# Proving SQL Queries Equivalent

## U-expressions, Q-expressions, and LIA*

**A from-scratch guide for people who want to *implement* these algorithms**


**Abstract**

Three systems dominate automated SQL equivalence checking under bag semantics: UDP (Chu et al., VLDB 2018), which introduced *U-expressions* and the U-semiring; QED (Wang, Pan and Cheung, VLDB 2024), which redesigned them into *Q-expressions*; and SQLSOLVER (Ding et al., SIGMOD 2024), which kept U-expressions and replaced the decision procedure with the theory LIA*. This document explains all three from first principles. We assume you know SQL and nothing else — no semiring theory, no SMT, no proof assistants. Every symbol is introduced with a SQL query that exhibits it, every algorithm is given as pseudocode you could type in, and every claim about the three systems is traced to a specific definition, theorem or table in the source papers. Plans are shown as Apache Calcite `RelNode` trees (the representation all three systems actually consume) with PostgreSQL plans as a secondary, executable cross-check.


**How to read this.** Section 2 builds the counting-function view of SQL and fixes a running example. Section 3 is U-expressions end to end. Section 4 is Q-expressions end to end. Section 5 is LIA*. Section 6 compares the three on examples chosen to separate them. Section 7 is an implementer's checklist. Appendix A collects every axiom on one page.

## Contents


**1 The problem, and why it is hard** **4**
- 1.1 What we are trying to decide . . . 4
- 1.2 Bag semantics, and why it changes everything . . . 4
- 1.3 The common architecture . . . 4
- 1.4 A warning about the published numbers . . . 5

**2 Preliminaries: a table is a counting function** **5**
- 2.1 The one idea everything rests on . . . 5
- 2.2 Query operators as arithmetic . . . 5
- 2.3 Σ versus Sum: two things called "sum" . . . 6
- 2.4 The running example: aggregate pushdown through a join . . . 6
  - 2.4.1 Calcite plans (primary) . . . 7
  - 2.4.2 Calcite applies the rule itself — and reveals the key fact . . . 7
  - 2.4.3 PostgreSQL plans (secondary cross-check) . . . 7
  - 2.4.4 Why the primary key is load-bearing . . . 7

**3 U-expressions: the U-semiring and UDP** **9**
- 3.1 The design goal, and the two constraints that shaped it . . . 9
- 3.2 The U-semiring . . . 9
- 3.3 Which structures actually *are* U-semirings? . . . 10
- 3.4 Translating SQL into U-expressions . . . 10
- 3.5 Sum-Product Normal Form (SPNF) . . . 11

3.6 Integrity constraints as identities . . . 12
3.7 A complete worked proof: the index rewrite . . . 12
3.8 The decision procedure . . . 13
3.9 Guarantees . . . 15
3.10 UDP on the running example — and why it fails . . . 15
3.11 Scorecard for U-expressions as of 2018 . . . 15

**4 Q-expressions and Qed** **16**
4.1 The one-line summary . . . 16
4.2 The concrete domain . . . 16
4.3 Query syntax . . . 17
4.4 Semantics . . . 17
4.5 Integrity constraints: rewrite the table, not the query . . . 18
4.6 NULL semantics . . . 18
4.7 Algorithm A: the complete fragment . . . 19
4.7.1 Scoped normal form (SNF) . . . 19
4.7.2 Linearisation (LNF) . . . 19
4.7.3 Unification and completeness . . . 20
4.8 Algorithm B: the general algorithm . . . 20
4.8.1 SPNF ([2] Alg. 4) . . . 20
4.8.2 Stabilisation ([2] Alg. 5) . . . 21
4.8.3 Unification ([2] Alg. 6) . . . 21
4.8.4 Encoding to first-order logic . . . 22
4.9 The running example, proved . . . 22
4.10 Results and honest limits . . . 23

**5 U-expressions + LIA*: SQLSolver** **24**
5.1 The diagnosis . . . 24
5.2 LIA and LIA* . . . 24
5.3 The algorithm, on Example 5.1 . . . 25
5.4 Three complications the textbook theory does not cover . . . 26
5.5 Extended U-expressions: modelling more SQL . . . 26
5.6 Ordered bag semantics (`ORDER BY`) . . . 27
5.7 Guarantees . . . 28
5.8 Results . . . 28
5.9 SQLSolver on the running example . . . 28
5.10 Scorecard . . . 29

**6 The three approaches compared** **29**
6.1 Where the effort went . . . 29
6.2 Capability matrix . . . 29
6.3 A gallery of separating examples . . . 29
6.4 The deep structural difference: squashed versus not . . . 31
6.5 On the published numbers . . . 31
6.6 What the combination would look like . . . 32

**7 An implementer's guide** **32**
7.1 Build order . . . 32
7.2 Data structures . . . 32
7.3 Ten traps, each of which will cost you a day . . . 33
7.4 Testing: use a real database . . . 33
7.5 Getting plans out of Calcite . . . 34

**A All axioms and definitions on one page** **34**

**B Symbol glossary** **35**

**C Reproducing the measurements** **35**

# 1 The problem, and why it is hard

## 1.1 What we are trying to decide

Given two SQL queries $Q_1$ and $Q_2$ over the same schema, decide whether they return the same answer *on every possible database instance*. Not on your data — on all data.

This matters because every relational engine contains a rule-based rewriter. Calcite, Spark SQL, CockroachDB and PostgreSQL all transform your query into a different one before executing it. If a rewrite rule is wrong, the engine silently returns wrong answers. Historically these bugs have been found by users, years later [1].

The general problem is undecidable. For queries with negation and arbitrary predicates you can encode first-order validity, so no algorithm decides all cases. Everything in this document is therefore a *sound but incomplete* procedure: when it says "equivalent" it is right; when it says "unknown" the queries may still be equivalent.

**The three answers a prover may give**

**EQUIVALENT** — proved, trust it. **NOT EQUIVALENT** — a counterexample database exists. **UNKNOWN** — the procedure ran out of power.

All three systems here answer EQUIVALENT or UNKNOWN. None of them is a counterexample finder. Refutation is a different problem, usually attacked by bounded model finding.

## 1.2 Bag semantics, and why it changes everything

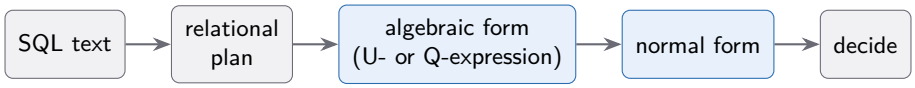
SQL tables are *bags* (multisets): duplicates are real and observable. `SELECT x FROM R` does not deduplicate. Two queries that agree as sets may disagree as bags:

```
SELECT x FROM R                 -- may return 1, 1, 2
SELECT DISTINCT x FROM R        -- returns    1, 2
```

The classical theory of query equivalence (containment mappings, the chase, homomorphism checking) is set-based. Under bags the picture is harder: bag *containment* of conjunctive queries is a long-standing open problem, while bag *equivalence* of unions of conjunctive queries is decidable [6]. That decidability result is the theoretical anchor for all three systems.

## 1.3 The common architecture

All three systems share a pipeline:




They differ in exactly two places, and it is worth stating this up front because everything else follows from it:

1. **The algebraic form.** UDP and SQLSolver use *U-expressions*, defined *axiomatically* over an abstract structure called a U-semiring. Qed uses *Q-expressions*, which are the same syntax pinned to a *concrete* semantic domain $\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}$.

2. **The decision step.** UDP compares normal forms *syntactically*. Qed encodes them into first-order logic and calls an SMT solver. SQLSolver encodes them into LIA*, a theory built for unbounded sums, and reduces that to plain LIA for an SMT solver.

| | algebra | decision step | one-line character |
|---|---|---|---|
| UDP 2018 | U-expression | syntactic isomorphism | the foundation; proves things by axiom-chasing |
| QED 2024 | Q-expression | SMT on FOL encoding | better *modelling*: NULLs, constraints, unknown operators |
| SQLSOLVER 2024 | U-expression | $\mathsf{LIA}^* \to \mathsf{LIA} \to$ SMT | better *arithmetic*: unbounded sums, real aggregates |

Table 1: The three systems at a glance. SPES [4], a fourth system, uses a *tree-structured* algebraic representation of relational operators rather than a U-expression, normalises it with hand-written rewrite rules, and calls an SMT solver only to compare predicates; we use it as a baseline throughout.

### 1.4 A warning about the published numbers

Every paper reports how many query pairs it proves on "the Calcite test suite". These numbers are *not* comparable across papers, because each group extracted its own pairs from a different Calcite version with a different filter. The clearest evidence: QED and SQLSOLVER both report SPES on "232 Calcite pairs" and get **95** and **99** respectively. Treat cross-paper percentages as indicative only. We return to this in Section 6.

## 2 Preliminaries: a table is a counting function

### 2.1 The one idea everything rests on

Write a bag not as a list of rows but as a function that counts:

**Definition you must internalise**
$R(t)$ = "how many copies of the row $t$ are in table $R$?"
A table of schema $\sigma$ is a function $\mathrm{Tuple}(\sigma) \to \mathbb{N}$, where $\mathrm{Tuple}(\sigma)$ is the set of *all* tuples of that schema — not just the ones present. Absent rows simply have count 0.

This is the *K-relation* view of Green, Karvounarakis and Tannen [5], with $K = \mathbb{N}$. Running example throughout:

$R$

| $x$ | $y$ |
|---|---|
| 1 | 10 |
| 1 | 20 |
| 2 | 30 |

$S$ (primary key on $a$)

| $a$ | $b$ |
|---|---|
| 1 | 100 |

so $R(1,10) = 1$, $R(1,20) = 1$, $R(2,30) = 1$, and $R(t) = 0$ for every other $t$; $S(1,100) = 1$ and $S(t) = 0$ elsewhere. Verified:

```
select count(*) from r where x = 1 and y = 10;    --> 1
select count(*) from r where x = 1 and y = 99;    --> 0
```

### 2.2 Query operators as arithmetic

Once tables are counting functions, the relational operators become arithmetic. This table is the Rosetta stone for the whole document; each line is derived properly in Section 3.

Two of these deserve immediate attention because beginners stumble on both.

| SQL | on counts | why |
|---|---|---|
| `FROM R` | $R(t)$ | by definition |
| `WHERE b` | $[b] \times R(t)$ | $[b]$ is 1 or 0; multiplying by 0 deletes the row |
| `R, S` (cross join) | $R(t_1) \times S(t_2)$ | every left row pairs with every right row |
| `UNION ALL` | $R(t) + S(t)$ | copies accumulate |
| `SELECT x` (projection) | $\sum_y R(x, y)$ | forgetting $y$ merges rows, so counts add |
| `DISTINCT` | $\|\|R(t)\|\|$ | clamp any count to 0 or 1 |
| `NOT EXISTS Q` | $\text{not}(\sum_t Q(t))$ | 1 exactly when the count is 0 |

Table 2: SQL as arithmetic on multiplicities.

**Why projection is a sum.** `SELECT x FROM R` throws away $y$. Two rows that differed only in $y$ become the same output row, so their counts must be *added*:

$$[\![\texttt{SELECT x FROM R}]\!](x) \;=\; \sum_y R(x, y).$$

Concretely $\sum_y R(1, y) = 1 + 1 = 2$, which is exactly `SELECT x, count(*) FROM r GROUP BY x` returning 2 for $x = 1$.

**Why the sum is "unbounded".** The index $y$ ranges over *every* value of its type, not over the rows of $R$. That looks alarming — an infinite sum — but almost every term is 0, because a table has finitely many rows. Widening the range cannot change the value:

```
select sum((select count(*) from r where r.x=1 and r.y=yv))
from generate_series(0,30) yv;          --> 2
-- same query over generate_series(-1000,1000)
                                        --> 2
```

Both UDP and QED keep unbounded summation as a *primitive* rather than proving finiteness, because proving that every operator preserves finiteness is painful and buys little [1, §3.1]. QED handles the pathological case by working in $\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}$ so that a genuinely infinite sum is still a well-defined value.

### 2.3 $\Sigma$ versus Sum: two things called "sum"

This trips up almost everyone, so we fix it now.

| | adds up | SQL | on the bag $\{10, 20\}$ |
|---|---|---|---|
| $\sum_y f(y)$ | multiplicities | `COUNT(*)` | $1 + 1 = 2$ |
| $\text{Sum}(\lambda y.\, f(y))$ | values $\times$ multiplicities | `SUM(y)` | $10{\cdot}1 + 20{\cdot}1 = 30$ |

Verified: `select count(*), sum(y) from r where x = 1` returns 2 and 30.

Here $\lambda y.\, f(y)$ is an anonymous function (Python's `lambda y:`). Because a bag *is* a counting function, writing $\lambda y.\, R(1, y)$ *is* writing the bag $\{10, 20\}$. Aggregates consume a whole bag, so their argument must be a function, not a number.

### 2.4 The running example: aggregate pushdown through a join

Everything in this document is illustrated on one rewrite — Calcite's `AggregateJoinTransposeRule`, which is also QED's motivating example [2, §2].

```
-- Q1 : aggregate ABOVE the join
select x, sum(y) as r from R join S on R.x = S.a group by x;
```

```
-- Q2 : aggregate BELOW the join
select t.x, t.r
from (select x, sum(y) as r from R group by x) t join S on t.x = S.a;
```

### 2.4.1 Calcite plans (primary)

These are real `RelNode` trees produced by Apache Calcite 1.19 with `R(X,Y)` and `S(A,B)` declared, `S.A` unique (`Statistics.of(rows, ImmutableList.of(ImmutableBitSet.of(0)))`):

```
-- Q1
LogicalAggregate(group=[{0}], SUMY=[SUM($1)])
  LogicalProject(X=[$0], Y=[$1])
    LogicalJoin(condition=[=($0,$2)],
                joinType=[inner])
      TableScan(table=[[R]])
      TableScan(table=[[S]])
```

```
-- Q2
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(condition=[=($0,$2)],
              joinType=[inner])
    LogicalAggregate(group=[{0}],
                     SUMY=[SUM($1)])
      TableScan(table=[[R]])
    TableScan(table=[[S]])
```

`LogicalAggregate` and `LogicalJoin` have swapped places. That is the entire rewrite.

### 2.4.2 Calcite applies the rule itself — and reveals the key fact

Running Calcite's own `AggregateJoinTransposeRule.EXTENDED` on $Q_1$ in a `HepPlanner` gives two very different answers depending on whether `S.A` is declared unique:

**`S.A` declared unique**

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(group=[{0}],
                     SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalProject(A=[$0])
      TableScan([[S]])
```

*exactly $Q_2$.*

**`S.A` *not* unique**

```
LogicalProject(X=[$0],
   $f4=[CAST(*($1, $3))])   <-- product!
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(group=[{0}],
                     SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalAggregate(group=[{0}],
                agg#0=[COUNT()])  <-- count!
      TableScan([[S]])
```

not $Q_2$: *the aggregate is multiplied by the number of matching `S` rows.*

**Read this twice — it is the whole document in one picture**

With the key, Calcite produces exactly $Q_2$. Without the key, Calcite *refuses* to produce $Q_2$; instead it counts the matching $S$ rows and **multiplies the aggregate by that count**. That compensating factor is literally the $\times S(a)$ term of the U-expression, discovered independently by a production optimiser. When we prove $Q_1 \equiv Q_2$ in Sections 3–5, the primary key is precisely what lets us delete that factor.

### 2.4.3 PostgreSQL plans (secondary cross-check)

PostgreSQL does *not* attempt this rewrite, so the two SQL texts execute as written — which makes it a clean way to observe both shapes running. On $10^6$ rows in `r` and $5 \cdot 10^4$ in `s`:

### 2.4.4 Why the primary key is load-bearing

Give $S$ a duplicate ($a = 1$ twice, no key) and run the same two queries on the same $R$:

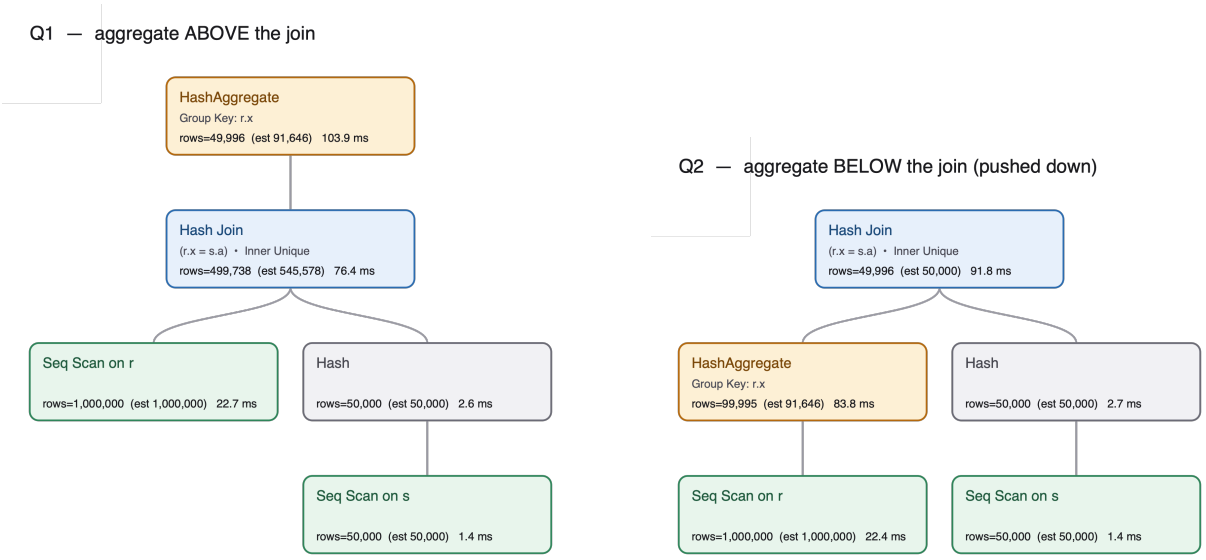



Figure 1: PostgreSQL 18 `EXPLAIN (ANALYZE, VERBOSE, BUFFERS)` for $Q_1$ (left) and $Q_2$ (right). Note `Inner Unique` on both joins: the planner read the primary key and recorded "at most one match on the inner side". *That flag is the same fact the provers write as* $\|S(a)\|$ — Postgres uses it to stop probing early, the provers use it to justify the rewrite.

| | $Q_1$ (agg above) | $Q_2$ (agg below) |
|---|---|---|
| rows entering the aggregate | 499,738 | 1,000,000 |
| groups produced | 49,996 | 99,995 |
| rows entering the join | 1,000,000 | 99,995 |
| final rows | 49,996 | 49,996 |
| execution time | 105.5 ms | 93.1 ms |

Table 3: Measured. The pushdown is only ≈12% faster here and its aggregate does *more* work; the win comes from the join seeing 99,995 rows instead of $10^6$. Whether to apply the rewrite is a cost question — entirely separate from whether it is *correct*, which is what this document is about.

| | result | rows |
|---|---|---|
| $Q_1$ (agg above join) | (1, 60) | 1 |
| $Q_2$ (agg below join) | (1, 30), (1, 30) | 2 |

Not equivalent, and wrong in two independent ways:

- $Q_1$ joins first, so every $R$ row is duplicated *before* the aggregate sees it. The bag entering `SUM` becomes $\{10, 10, 20, 20\}$ instead of $\{10, 20\}$, so the **value** is corrupted: 60 instead of 30.
- $Q_2$ aggregates first, so `SUM` sees the clean bag. The duplication hits the **output row count** instead: two copies of $(1, 30)$.

This is the same phenomenon Calcite compensated for in §2.4.2: with count = 2, Calcite's rewritten plan computes $30 \times 2 = 60$, matching $Q_1$. On PostgreSQL, `Inner Unique` disappears from $Q_1$'s plan the moment the key is dropped.

**The specification of the running example**

$Q_1 \equiv Q_2$ **if and only if** the join column on the right is distinct. A prover that cannot turn "`a` is a primary key" into a usable algebraic fact cannot prove this rewrite; one that proves it *without* using the key is unsound.

# 3 U-expressions: the U-semiring and UDP

*Source: Chu, Murphy, Roesch, Cheung, Suciu, PVLDB 11(11), 2018 [1].*

## 3.1 The design goal, and the two constraints that shaped it

The authors wanted an algebra in which SQL equivalence proofs are *machine-checkable down to axioms*, so they could be discharged in Coq. That imposes two constraints that explain almost every design choice:

1. **Everything must be an identity** (an equation $E_1 = E_2$), not a logical sentence like $\forall t\,(\phi \Rightarrow \psi)$. Identities can be used as rewrite rules; implications need a theorem prover. This is why the key constraint gets the unusual definition in §3.6.
2. **As few axioms as possible.** Fewer axioms means a smaller trusted base and a simpler proof search. The paper deliberately *excludes* an order relation $\leq$, even though $\leq$ would make keys easy to state.

## 3.2 The U-semiring

**Definition 3.1** (U-semiring, [1] Def. 3.1)**.** An *unbounded semiring* is a structure $(\mathcal{U}, 0, 1, +, \times, \|\cdot\|, \mathrm{not}(\cdot), (\sum_D)_D$ where $(\mathcal{U}, 0, 1, +, \times)$ is a commutative semiring and the extra operations satisfy the axioms below.

A commutative semiring is just "arithmetic without subtraction": $+$ and $\times$ are associative and commutative, $0$ is the unit of $+$, $1$ the unit of $\times$, $\times$ distributes over $+$, and $0 \times x = 0$.

**Squash $\|\cdot\|$ (models `DISTINCT`).**

$$\|0\| = 0, \qquad \|1 + x\| = 1 \tag{1}$$
$$\|\|x\| + y\| = \|x + y\| \tag{2}$$
$$\|x\| \times \|y\| = \|x \times y\| \tag{3}$$
$$\|x\| \times \|x\| = \|x\| \tag{4}$$
$$x \times \|x\| = x \tag{5}$$
$$x^2 = x \;\Rightarrow\; \|x\| = x \tag{6}$$

**Negation $\mathrm{not}(\cdot)$ (models `NOT EXISTS`).**

$$\mathrm{not}(0) = 1 \tag{7}$$
$$\mathrm{not}(x \times y) = \|\mathrm{not}(x) + \mathrm{not}(y)\| \tag{8}$$
$$\mathrm{not}(x + y) = \mathrm{not}(x) \times \mathrm{not}(y) \tag{9}$$
$$\mathrm{not}(\|x\|) = \|\mathrm{not}(x)\| = \mathrm{not}(x) \tag{10}$$

**Unbounded summation $\sum_t$ (models projection).**

$$\sum_t (f_1(t) + f_2(t)) = \sum_t f_1(t) + \sum_t f_2(t) \tag{11}$$

$$\sum_{t_1} \sum_{t_2} f(t_1, t_2) = \sum_{t_2} \sum_{t_1} f(t_1, t_2) \tag{12}$$

$$x \times \sum_t f(t) = \sum_t (x \times f(t)) \tag{13}$$

$$\left\| \sum_t f(t) \right\| = \left\| \sum_t \|f(t)\| \right\| \tag{14}$$

**Predicates and equality.** For each SQL predicate $b$ there is $[b] \in \mathcal{U}$ with

$$[b] = \|[b]\| \tag{15}$$

and equality satisfies *excluded middle*, *substitution of equals* and *uniqueness*:

$$[e_1 = e_2] + [e_1 \neq e_2] = 1 \tag{16}$$

$$f(e_1) \times [e_1 = e_2] = f(e_2) \times [e_1 = e_2] \tag{17}$$

$$\sum_t [t = e] = 1 \tag{18}$$

**Lemma 3.2** (the workhorse, [1] Eq. 15)**.** $\sum_t [t = e] \times f(t) = f(e)$.

*Proof.* By (17), $\sum_t [t = e] f(t) = \sum_t [t = e] f(e)$; by (13) pull $f(e)$ out, giving $f(e) \times \sum_t [t = e]$; by (18) the sum is 1. □

**Lemma 3.2 is the single most-used rule in the whole system**
It says: *a summation whose body pins the summation variable to a value collapses to that one value.* A join condition `ON x = a` creates exactly such a pin, which is why join conditions make summations disappear. QED rediscovers this as "stabilisation" (§4.8.2) but finds the pin with an SMT solver instead of syntactically.

### 3.3 Which structures actually *are* U-semirings?

This matters for soundness. The paper gives four models [1, §3.1]: $\mathbb{N}$ (if all summation domains are finite); $\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}$ (for arbitrary domains, with $x + \infty = \infty$, $0 \times \infty = 0$, $x \times \infty = \infty$ for $x \neq 0$); univalent types; and cardinal numbers.

$\overline{\mathbb{N}}$ is the one to keep in mind — and note that QED later takes $\overline{\mathbb{N}}$ as its *only* model rather than as one model among many. That single decision is most of the difference between the two systems.

*Remark* 3.3 (why no order relation). One could define a key by "every key value occurs at most once", i.e. $\sum_t R(t) \times [t.k = e] \leq 1$. But $\leq$ requires an ordered semiring, adding axioms. The paper instead finds an *equational* formulation (§3.6). Beware: axioms that look obviously true may not be. The paper's counterexample is the semiring of diagonal $2 \times 2$ matrices over $\overline{\mathbb{N}}$, in which the plausible rule "if $x \neq 0$ then $\|x\| = 1$" fails.

### 3.4 Translating SQL into U-expressions

Every query $q$ becomes a function $[\![q]\!] : \text{Tuple}(\sigma) \to \mathcal{U}$.

$$
\begin{aligned}
[\![R]\!] &= \lambda t.\, R(t) \\
[\![\texttt{SELECT * FROM R x, S y}]\!] &= \lambda (t_1, t_2).\, R(t_1) \times S(t_2) \\
[\![\texttt{SELECT * FROM R WHERE b}]\!] &= \lambda t.\, [\, b \,] \times R(t) \\
[\![\texttt{SELECT a FROM R}]\!] &= \lambda t.\, \sum_{t'} [\, t'.a = t \,] \times R(t') \\
[\![\texttt{q1 UNION ALL q2}]\!] &= \lambda t.\, [\![q_1]\!](t) + [\![q_2]\!](t) \\
[\![\texttt{DISTINCT q}]\!] &= \lambda t.\, \|[\![q]\!](t)\| \\
[\![\texttt{EXISTS q}]\!] &= \|[\![q]\!]\|\,, \qquad [\![\texttt{NOT EXISTS q}]\!] = \mathrm{not}([\![q]\!])
\end{aligned}
$$

Boolean connectives follow the same pattern: `AND` $\mapsto \times$, `OR` $\mapsto +$, `NOT` $\mapsto \mathrm{not}(\cdot)$.

**Aggregates.** UDP treats every aggregate as an *uninterpreted function* applied to a U-expression: $\mathrm{agg}(E)$ is a value expression usable inside a predicate, e.g. $[\,\mathrm{agg}(E) = t.a\,]$. Nothing is known about it except congruence: equal arguments give equal results. `GROUP BY` is de-sugared into a correlated subquery [1, §3.2]:

```
SELECT x.k AS k, agg(x.a) AS a1 FROM R x GROUP BY x.k
-- becomes
SELECT y.k AS k,
       agg(SELECT x.a AS a FROM R x WHERE x.k = y.k) AS a1
FROM R y
```

*Remark* 3.4 (an implementation trap). As printed above (and in the paper), the outer query iterates over *every* row of `R`, so a key occurring twice yields two output rows — but `GROUP BY` emits one row per *distinct* key. A faithful de-sugaring needs `SELECT DISTINCT` on the outer query, i.e. a squash. QED makes this explicit in its `GroupBy` rule (§4.4), where the group-non-empty factor is wrapped in $\|\cdot\|$. If you implement the de-sugaring literally you will get wrong multiplicities.

### 3.5 Sum-Product Normal Form (SPNF)

**Definition 3.5** (SPNF, [1] Def. 3.3)**.** $E$ is in SPNF if $E = T_1 + \cdots + T_n$ and each term has the shape

$$
T \;=\; \sum_{t_1,\ldots,t_m} [\, b_1 \,] \times \cdots \times [\, b_k \,] \;\times\; \|E_s\| \;\times\; \mathrm{not}(E_n) \;\times\; M_1 \times \cdots \times M_j
$$

where each $M_i$ is a table application $R(t)$, each $[\, b_i \,]$ a predicate, **exactly one** squash factor $\|E_s\|$ (omitted when $E_s = 1$), **exactly one** negation factor $\mathrm{not}(E_n)$ (omitted when $E_n = 0$), and $E_s, E_n$ are themselves in SPNF.

Think of SPNF as "disjunctive normal form for bags": a sum of products, all summations hoisted to the front, all predicates to the left, all squashes fused into one.

**Theorem 3.6** ([1] Thm. 3.4)**.** *Every U-expression can be rewritten into SPNF using only U-semiring axioms.*

The rewrite system, applied left-to-right until fixpoint:

$$
\begin{aligned}
E_1 \times (E_2 + E_3) &\rightsquigarrow E_1E_2 + E_1E_3 & \qquad (E_1 + E_2) \times E_3 &\rightsquigarrow E_1E_3 + E_2E_3 \\
E_1 \times (E_2 \times E_3) &\rightsquigarrow (E_1 \times E_2) \times E_3 & \qquad \cdots M[\, b \,] \cdots &\rightsquigarrow \cdots [\, b \,] M \cdots \\
\sum_t (f_1 + f_2) &\rightsquigarrow \sum_t f_1 + \sum_t f_2 & \qquad E \times \sum_t f(t) &\rightsquigarrow \sum_t E \times f(t) \\
\Big(\sum_t f(t)\Big) \times E &\rightsquigarrow \sum_t f(t) \times E & \qquad \|E_1\| \times \|E_2\| &\rightsquigarrow \|E_1 \times E_2\| \\
\mathrm{not}(E_1) \times \mathrm{not}(E_2) &\rightsquigarrow \mathrm{not}(E_1 + E_2)
\end{aligned}
$$

Each rule is an axiom used in one direction; each strictly makes progress, so the system terminates. Note how (3) and (9) are exactly what let you *merge* multiple squashes and negations into the single ones Definition 3.5 demands.

### 3.6 Integrity constraints as identities

This is UDP's most original contribution and the part most worth stealing.

**Definition 3.7** (KEY, [1] Def. 4.1)**.** $R.k$ is a key iff for all $t, t'$:

$$[\,t.k = t'.k\,] \times R(t) \times R(t') \;=\; [\,t = t'\,] \times R(t).$$

Why this is the right definition, in two steps. Set $t = t'$: the left becomes $R(t) \times R(t)$ and the right $R(t)$, so $R(t)^2 = R(t)$, which over $\mathbb{N}$ forces $R(t) \in \{0, 1\}$ — *no duplicate rows.* Now take $t \neq t'$ with $t.k = t'.k$: the right side is 0, so $R(t) \times R(t') = 0$, i.e. at most one of them is present — *no two rows share a key.* Together:

**Theorem 3.8** ([1] Thm. 4.2)**.** *Over the U-semiring* $\mathbb{N}$*, Definition 3.7 holds iff* $R.k$ *is a key in the ordinary sense.*

**Definition 3.9** (FOREIGN KEY, [1] Def. 4.4)**.** $S.k'$ references $R.k$ iff $S(t') = S(t') \times \sum_t R(t) \times [\,t.k = t'.k'\,]$.

Read it as: "if $t'$ is in $S$ then multiplying by the number of matching $R$ rows changes nothing", which over $\mathbb{N}$ forces that number to be exactly 1 whenever $S(t') \neq 0$.

**Theorem 3.10** (key implies squash-stability, [1] Thm. 4.3)**.** *If* $R.k$ *is a key then for any* $E$*,* $e$*,* $b$*:*

$$\sum_t [\,b\,] \times \|E\| \times [\,t.k = e\,] \times R(t) = \Big\| \sum_t [\,b\,] \times \|E\| \times [\,t.k = e\,] \times R(t) \Big\|.$$

*Proof sketch.* By (6) it suffices to show the left side squared equals itself. Multiply Definition 3.7 through by $[\,b\,]^2 \|E\|^2$ (using $[\,b\,]^2 = [\,b\,]$ and $\|E\|^2 = \|E\|$), sum over $t$ and $t'$, apply (13) twice on the left and Lemma 3.2 on the right. □

**Why Theorem 3.10 matters:** it is the formal statement of "a lookup by key returns at most one row". It is what turns a key into a squash, and it is the ancestor of QED's much simpler rule $S \rightsquigarrow \lambda a.\ \|S(a)\|$.

**Views and indexes.** A view becomes an assertion $v(t) = q(t)$ and is inlined. An index on $R.a$ is modelled, GMAP-style, as the view `SELECT x.a, x.k FROM R x` where $k$ is the key — then inlined too. So indexes need no new machinery.

### 3.7 A complete worked proof: the index rewrite

This is [1, Ex. 4.7], and it is the best single illustration of axiom-chasing. $R$ has key $k$; $I$ is an index on $R.a$.

```
-- Q1 : plain scan
SELECT * FROM R WHERE R.a >= 12
-- Q2 : index lookup
SELECT t2.* FROM I t1, R t2 WHERE t1.k = t2.k AND t1.a >= 12
```

$[\![Q_1]\!](t) = R(t) \times [\,t.a \geq 12\,]$. Inlining $I$ and normalising to SPNF:

$$[\![Q_2]\!](t) = \sum_{t_1, t_2, t_3} [\,t_2 = t\,][\,t_1.k = t_2.k\,][\,t_1.a \geq 12\,][\,t_3.k = t_1.k\,][\,t_3.a = t_1.a\,] R(t_3) R(t_2).$$

1. $t_1$ ranges over tuples of $I$'s schema, namely $(k, a)$. The two equalities $[t_3.k = t_1.k]$ and $[t_3.a = t_1.a]$ therefore *determine* $t_1$ completely: $[t_1 = (t_3.k, t_3.a)]$. Apply Lemma 3.2 to delete $\sum_{t_1}$ and substitute:

$$\sum_{t_2,t_3} [t_2 = t][t_3.k = t_2.k][t_3.a \geq 12]R(t_3)R(t_2).$$

2. Apply the KEY identity (Def. 3.7) to $[t_3.k = t_2.k]R(t_3)R(t_2) = [t_2 = t_3]R(t_2)$:

$$\sum_{t_2,t_3} [t_2 = t][t_3.a \geq 12]R(t_2)[t_2 = t_3].$$

3. Lemma 3.2 on $t_3$ (pinned to $t_2$), then on $t_2$ (pinned to $t$):

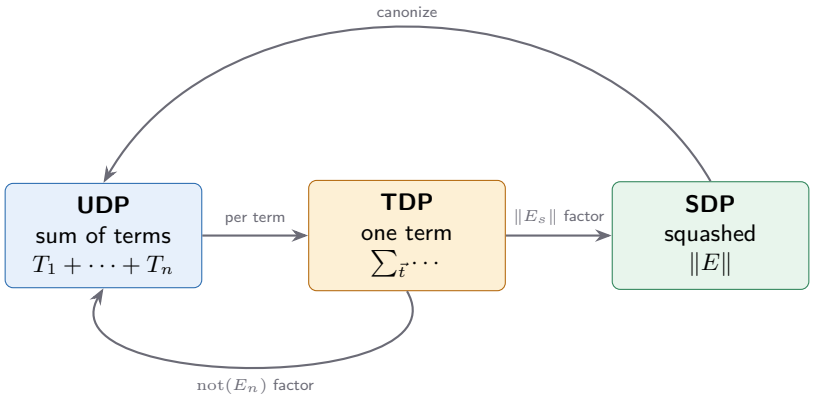
$$[\![Q_2]\!](t) = [t.a \geq 12] \times R(t) = [\![Q_1]\!](t). \qquad \blacksquare$$

**The proof strategy in one sentence**

Put both queries in SPNF, then repeatedly (i) delete summations whose variable is pinned, and (ii) apply constraint identities, until the two expressions are *syntactically identical*.

### 3.8 The decision procedure

UDP is three mutually recursive procedures. Read them together with Definition 3.5; each procedure handles one layer of that grammar.




**Step 0: canonize ([1] Alg. 1).** Given $E$ in SPNF and constraint set $C$:

1. Close equality predicates under transitivity: $[e_1{=}e_2][e_2{=}e_3] = [e_1{=}e_2][e_2{=}e_3][e_1{=}e_3]$.
2. Repeatedly apply Lemma 3.2 to delete pinned summations.
3. For each key constraint, rewrite with Def. 3.7 and Thm. 3.10.
4. For each foreign key, rewrite with Def. 3.9.

**Canonization may not terminate**

Steps 3–4 are a *chase*. The paper says so explicitly: a cycle in the key/foreign-key graph can loop forever. Their mitigation is pragmatic — step 4 is applied to squashed expressions only until the result stops changing (checked by calling SDP), and elsewhere only until no new base relation is introduced. If you implement this, put a depth bound on it.

```
procedure UDP(E1, E2, C):
    E1 <- canonize(E1, C);  E2 <- canonize(E2, C)
    # E1 = T(1,1)+...+T(1,n),  E2 = T(2,1)+...+T(2,m)
    if m != n: return false                  # <-- the hard ceiling
    for each permutation p of E1's terms:
        if for all i: TDP(p[i], T(2,i), C): return true
    return false
```

**Step 2: TDP ([1] Alg. 3).** Two terms are compared by searching for a *bijection* $h$ between their summation variables; after substituting, the terms must match componentwise:

- **predicates:** $[b_{1,1}]\cdots[b_{1,k}] = [b_{2,1}]\cdots[b_{2,m}]$, decided by *congruence closure* — compute equivalence classes of variables and function applications and compare. E.g. $[a{=}b][c{=}d][b{=}e][f(a){=}g(d)]$ matches $[a{=}b][a{=}e][c{=}d][f(e){=}g(c)]$ via the classes $\{a,b,e\},\{c,d\},\{f(a),f(e)\},\{g(c),g(d)\}$. Aggregates appearing in predicates are treated as uninterpreted functions here.
- **squash factors:** $\|E_{1,1}\|$ vs $\|E_{2,1}\|$ by SDP;
- **negation factors:** by calling UDP recursively;
- **table factors:** same number, and $M_{1,1},\ldots,M_{1,j}$ *identical* to $M_{2,1},\ldots,M_{2,\ell}$.

> **Where UDP is weakest**
> The predicate check is congruence closure over *uninterpreted* functions. It proves $[a{=}b]\Rightarrow[f(a){=}f(b)]$ but it does *not* know arithmetic. So $[a+b=0]$ and $[a=-b]$ are different predicates to UDP, and it fails. This one limitation motivates both successors: SPES and QED call an SMT solver here instead.

**Step 3: SDP ([1] Alg. 4).** Under a squash, multiplicities do not matter, so *set* reasoning applies. SDP first flattens nested squashes using

**Lemma 3.11** ([1] Lem. 5.1)**.** $\|a\times\|x\|+y\| = \|a\times x+y\|$ *in any U-semiring.*

then canonizes both sides and checks mutual containment — but without an order relation it cannot express containment directly. Instead it *minimises* each term using only U-semiring axioms (the analogue of conjunctive-query minimisation) and compares the minimal forms syntactically:

$$\forall i\,\exists j.\ \text{minimize}(\|T_i\|) = \text{minimize}(\|T'_j\|)\ \wedge\ \forall j\,\exists i.\ \text{minimize}(\|T'_j\|) = \text{minimize}(\|T_i\|).$$

**Example 3.12** ([1] Ex. 5.2 — self-join under DISTINCT)**.** `SELECT DISTINCT x.a FROM R x, R y` ≡ `SELECT DISTINCT R.a FROM R`.

$$Q_1(t) = \Big\|\sum_{t_1,t_2}[t_1.a=t]R(t_1)R(t_2)\Big\|$$

Insert $1=[t_1{=}t_2]+[t_1{\neq}t_2]$ by (16) and distribute:

$$Q_1(t) = \Big\|\sum_{t_1}[t_1.a{=}t]R(t_1)R(t_1) + \sum_{t_1,t_2}[t_1{\neq}t_2][t_1.a{=}t]R(t_1)R(t_2)\Big\|$$

(the first sum used Lemma 3.2 on $t_2$). By (14) and (4), $R(t_1)R(t_1)$ under a squash becomes $R(t_1)$. Factor with (11)/(13):

$$Q_1(t) = \Big\|\sum_{t_1}[t_1.a{=}t]R(t_1)\times\Big(1+\sum_{t_2}[t_1{\neq}t_2]R(t_2)\Big)\Big\|$$

and by (1) ($\|1+x\|=1$) together with (3), the parenthesised factor collapses, giving $\|\sum_{t_1}[t_1.a=t]R(t_1)\| = Q_2(t)$.

### 3.9 Guarantees

**Theorem 3.13** (soundness, [1] Thm. 5.3)**.** *If UDP returns true, the queries are equivalent under standard SQL semantics.*

The argument is short because every step is an axiom application, and $\mathbb{N}$ is a model of the axioms.

**Theorem 3.14** (completeness for UCQ)**.** *UDP is complete when both inputs are unions of conjunctive queries, evaluated under set semantics only, or under bag semantics only.*

For a bag CQ `SELECT p FROM R1 t1,...,Rn tn WHERE b` with $b$ a conjunction of equalities, canonization yields the *unique* canonical form $\sum_{t_1 \ldots t_n} [\, b \,] R_1(t_1) \cdots R_n(t_n)$; for the set version the same inside a squash. Uniqueness plus the term-matching search gives completeness.

*Remark* 3.15 (a genuine incompleteness, and it is interesting). U-equivalence is *stronger* than SQL equivalence, because U-semirings include models with infinite relations. Take a first-order *infinity axiom* $\varphi$ — a sentence false on every finite $R$ but satisfiable on an infinite one (e.g. "$R$ is a non-empty, injective, non-surjective function"). Encode $\varphi$ as a query $Q$ returning one row when $\varphi$ holds and none otherwise. On every finite database $Q$ is equivalent to the empty query, but they are *not* U-equivalent, because they differ in the model $\overline{\mathbb{N}}$ with infinite support. So UDP must answer UNKNOWN. This is the price of keeping unbounded summation primitive.

### 3.10 UDP on the running example — and why it fails

Take $Q_1, Q_2$ from §2.4. Two things go wrong, and both are structural.

**(a) The aggregate is a black box compared out of context.** After de-sugaring, both queries contain Sum applied to a U-expression. UDP can conclude $\mathrm{Sum}(E_1) = \mathrm{Sum}(E_2)$ only by congruence, i.e. only if $E_1$ and $E_2$ are equal *as U-expressions*. Here

$$E_1 = \lambda y.\, R(x, y) \times \|S(x)\| \qquad \text{versus} \qquad E_2 = \lambda y.\, R(x, y),$$

and these are genuinely *not* equal: when $x$ does not occur in $S$, the left is 0 and the right is not. The recursive check is performed with no knowledge of the enclosing context, so it fails — correctly, but uselessly. Fixing this is precisely QED's $\phi_1 \vee \phi_2$ trick (§4.8.3).

**(b) Predicate matching is syntactic.** Even the easy part — deleting $\sum_a$ using $[\, x = a \,]$ — only works when the pin is syntactically visible. Write the join condition as `x - a = 0` and UDP no longer sees it.

**What UDP *can* do here.** It handles the primary key fine: Theorem 3.10 converts the key lookup into a squash, exactly as needed. The key machinery is not the weak point — the aggregate and the syntactic predicate check are.

### 3.11 Scorecard for U-expressions as of 2018

| | |
|---|---|
| **Strong** | minimal axiom set, machine-checkable proofs, first equational treatment of keys/foreign keys, UCQ-complete under set *or* bag semantics, handles mixed set/bag queries and `NOT EXISTS`. |
| **Weak** | syntactic predicate matching (no arithmetic); aggregates uninterpreted and compared out of context; `UDP` bails when the two sides have different numbers of terms; canonization may not terminate; no NULL semantics at all, hence no outer joins; proved 33–34 of the 232 Calcite pairs. |

Both successors attack exactly one of these lists. Qed attacks "no NULLs, no SMT, context-free subquery checks". SQLSolver attacks "different numbers of terms, uninterpreted aggregates".

# 4 Q-expressions and Qed

*Source: Wang, Pan, Cheung, PVLDB 17(11):3602–3614, 2024 [2].*

## 4.1 The one-line summary

Q-expressions have *the same syntax* as U-expressions. What changes is that they are pinned to a single concrete semantic domain, which makes every operator *definable* rather than axiomatised, which in turn makes them cheaply encodable into SMT. Everything else in Qed follows from that.

| | **U-expression** | **Q-expression** |
|---|---|---|
| semantics | any U-semiring; operators fixed by *axioms* | the single model $\overline{\mathbb{N}}$; operators fixed by *definitions* |
| $\|\|a\|\|$ | 6 axiom lines (7 identities) | *defined* as $[\,a \neq 0\,]$ |
| not$(a)$ | 4 axiom lines | *defined* as $[\,a = 0\,]$ |
| decision | syntactic isomorphism | SMT, used at every stage |
| NULL | absent | first class (option types, 3-valued logic) |
| unknown operators | unsupported | QOp/HOp, first class |

## 4.2 The concrete domain

**Definition 4.1** ([2] Eq. 4)**.** Multiplicities live in $\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}$ with

$$\infty + a = \infty, \qquad 0 \times \infty = 0, \qquad a \times \infty = \infty \ (a \neq 0),$$

$$\sum_s f(s) = \begin{cases} f(s_1) + \cdots + f(s_n) & \text{if } \mathrm{supp}(f) = \{s_1, \ldots, s_n\} \text{ is finite} \\ \infty & \text{otherwise,} \end{cases}$$

$$[\,P\,] = \begin{cases} 1 & P \text{ true} \\ 0 & \text{else,} \end{cases} \qquad \|a\| = [\,a \neq 0\,], \qquad \mathrm{not}(a) = [\,a = 0\,].$$

A *Q-expression* is any expression of type $\overline{\mathbb{N}}$.

**Why this is more than bookkeeping**

$\overline{\mathbb{N}}$ *is* a U-semiring, so nothing is lost. But because $\|a\|$ is now literally $[\,a \neq 0\,]$, a squash of a table application is just a Boolean — an uninterpreted predicate an SMT solver can reason about directly. In the axiomatic setting $\|\cdot\|$ is an opaque symbol you may only rewrite with the six squash axioms. This is the difference between "call Z3" and "search for an axiom chain".

### 4.3 Query syntax

QED fixes a small relational core [2, Fig. 2]:

$$\begin{aligned} Q ::= \ & \texttt{Table}(R:\sigma) && \mid \texttt{Values}(v_1,\ldots,v_n) \\ \mid \ & \texttt{Filter}(P,Q) && \mid \texttt{Proj}(f,Q) \\ \mid \ & \texttt{Join}(Q_1,Q_2) && \mid \texttt{Union}(Q_1,Q_2) \\ \mid \ & \texttt{Minus}(Q_1,Q_2) && \mid \texttt{Distinct}(Q) \\ \mid \ & \texttt{GroupBy}(k,\alpha(f),Q) && \mid \mathrm{QOp}(o,v_1,\ldots,v_n,Q) \\ v,P,f ::= \ & x \mid \texttt{Exists}(Q) && \mid \texttt{Op}(o,\vec{v}) \mid \mathrm{HOp}(o,\vec{v},Q) \end{aligned}$$

Two entries deserve comment.

QOp and HOp are **uninterpreted operators** — *first-class* escape hatches. `LIMIT 10` is $\mathrm{QOp}(\texttt{Limit},10,Q)$; `a IN Q` is $\mathrm{HOp}(\texttt{In},a,Q)$. Anything QED does not model becomes one of these and still participates in reasoning through congruence, rather than aborting the proof.

*Remark* 4.2 (what is deliberately missing). Set `Minus` and set `Intersect` are modelled; *bag* minus and *bag* intersect are not. The paper's footnote is explicit: those would need a *monus* (truncated subtraction) at the semiring level, which QED declines to add. Handle them as QOp.

### 4.4 Semantics

$$\begin{aligned} [\![\texttt{Table}(R{:}\sigma)]\!] &= \lambda x.\, R(x) \\ [\![\texttt{Values}(v_1..v_n)]\!] &= \lambda x.\, [\,x{=}v_1\,] + \cdots + [\,x{=}v_n\,] \\ [\![\texttt{Filter}(P,Q)]\!] &= \lambda x.\, [\,[\![P]\!](x) = \mathsf{True}\,] \times [\![Q]\!](x) \\ [\![\texttt{Proj}(f,Q)]\!] &= \lambda x.\, \sum_s [\,x = [\![f]\!](s)\,] \times [\![Q]\!](s) \\ [\![\texttt{Join}(Q_1,Q_2)]\!] &= \lambda x_1,x_2.\, [\![Q_1]\!](x_1) \times [\![Q_2]\!](x_2) \\ [\![\texttt{Union}(Q_1,Q_2)]\!] &= \lambda x.\, [\![Q_1]\!](x) + [\![Q_2]\!](x) \\ [\![\texttt{Distinct}(Q)]\!] &= \lambda x.\, \|[\![Q]\!](x)\| \\ [\![\texttt{Minus}(Q_1,Q_2)]\!] &= \lambda x.\, \|Q_1(x) \times \mathrm{not}(Q_2(x))\| \\ [\![\texttt{GroupBy}(k,\alpha(f),Q)]\!] &= \lambda x,y.\, \underbrace{\Big\|\sum_s [\,x = [\![k]\!](s)\,] \times [\![Q]\!](s)\Big\|}_{\text{the group is non-empty}} \times \underbrace{\Big[\,y = \mathrm{HOp}(\alpha, \lambda y'.\, \sum_s [\![Q]\!](s) \times [\,x = [\![k]\!](s) \wedge y' = [\![}_{y \text{ is the aggregate of the group's bag}} \end{aligned}$$

and for scalars, $[\![\texttt{Exists}(Q)]\!] = \|\sum_s [\![Q]\!](s)\|$.

**Read the `GroupBy` rule slowly — it is the crux of the paper.** The output row is the pair $(x,y)$ = (group key, aggregate value). Two factors:

- The *squashed* sum says "at least one input row has key $x$", and the squash is what makes `GROUP BY` emit **one** row per key regardless of input multiplicities. (This is exactly the subtlety UDP's de-sugaring glosses over — Remark 3.4.)
- The bracket pins $y$ to $\alpha$ applied to a *bag*: the function $\lambda y'.\ldots$ whose value at $y'$ is the multiplicity of $y'$ within the group. $\alpha$ is an HOp, i.e. **uninterpreted**. QED knows only that equal bags give equal aggregates.

**Definition 4.3** (the equivalence problem)**.** Write $\Gamma \mid \Delta \mid \Phi \vdash Q_1 \stackrel{?}{=} Q_2$, where $\Gamma$ are table symbols, $\Delta$ other uninterpreted symbols and $\Phi$ an assumed predicate. It asks whether $[\![Q_1]\!](s) = [\![Q_2]\!](s)$ for all instantiations of $\Gamma,\Delta$ satisfying $\Phi$ and all rows $s$.

### 4.5 Integrity constraints: rewrite the table, not the query

**Primary key ([2] Eqs. 6–8).** Let $R$ have schema $K \times S$ with key column(s) $k$. A table with a primary key is *exactly* a subset of $K$ plus a function $f_R : K \to S$ giving the non-key columns. Formally, for any predicate $P$ over tables,

$$\forall R \text{ with primary key } k.\ P(R) \iff \forall R', f_R.\ P(\lambda k, s.\ \|R'(k)\| \times [\, s = f_R(k)\,]) \tag{19}$$

which licenses the rewrite

$$R \rightsquigarrow \lambda k, s.\ \|R'(k)\| \times [\, s = f_R(k)\,] \tag{20}$$

introducing a fresh table symbol $R'$ of schema $K$ and a fresh function symbol $f_R$. When $k$ is the *only* column this collapses to the form everyone quotes:

$$R \rightsquigarrow \lambda k.\ \|R(k)\|\,. \tag{21}$$

> **Do not quote Eq. (21) when the table has non-key columns**
> In our running example $S$ has schema $(a, b)$ with key $a$, so the applicable rule is (20): $S \rightsquigarrow \lambda a, b.\ \|S'(a)\| \times [\, b = f_S(a)\,]$. The familiar $\lambda a.\ \|S(a)\|$ appears only after $b$ is projected away — which the queries do, since neither mentions $S.b$. The distinction matters the moment a query reads a non-key column.

The word *complete* in the paper is Eq. (19): the rewrite yields an equivalence problem that is equivalent to the original, not merely implied by it. Contrast SPES, whose primary-key support is a finite list of query-level rewrite rules — sound, but only firing on shapes someone anticipated.

**Foreign key ([2] Eq. 9).**

$$S \rightsquigarrow S, R' \mid \forall b, k.\ \|S(b, k)\| \to \|R'(k)\| \vdash S$$

Note what changed relative to UDP: UDP applies its foreign-key identity *eagerly* as a rewrite, which does not terminate when two tables reference each other. QED emits *one global logical formula* instead and lets the SMT solver unfold its consequences lazily. That is a strictly better engineering decision and removes UDP's non-termination hazard.

**Check constraint.** $\forall s.\ \|R(s)\| \to C(s)$. One line, same idea.

### 4.6 NULL semantics

UDP has none; this alone rules out outer joins. QED lifts every type $T$ to Option$(T)$, an algebraic datatype modern SMT solvers support natively.

- **Strict by default:** an $n$-ary operation returns Null exactly when some argument is Null. This lifts $+$, $\times$, comparisons, etc.
- **Three-valued `AND`/`OR`** are the documented exceptions:

$$a \texttt{ And } b := \text{ite}(a = \text{Null},\ \text{ite}(b = \text{True}, \text{Null}, b),\ \text{ite}(a = \text{True}, b, a))$$
$$a \texttt{ Or } b := \text{ite}(a = \text{Null},\ \text{ite}(b = \text{False}, \text{Null}, b),\ \text{ite}(a = \text{False}, b, a))$$

  Check them: Null `And` False $=$ False (not Null), and Null `Or` True $=$ True. Strictness would get both wrong.

- **Quantified comparison:**

  $a \,\langle\mathrm{cmp}\rangle\, \mathsf{Some}\, Q \coloneqq \mathrm{ite}(\exists x.\, \|Q(x)\| \wedge (x \,\langle\mathrm{cmp}\rangle\, a) = \mathsf{True},\ \mathsf{True},\ \mathrm{ite}(\exists x.\, \|Q(x)\| \wedge (x \,\langle\mathrm{cmp}\rangle\, a) = \mathsf{Null},\ \mathsf{Null},\ \mathsf{Fals}$

  `a IN Q` is the special case with $\langle\mathrm{cmp}\rangle$ being `=`. The nesting order encodes SQL's rule: *a true match wins over a null one*, and only if there is no match at all and no null do you get False.

- **`COUNT(*)` vs `COUNT(k)`:** modelled by optionally applying a non-Null filter before the aggregate.

## 4.7 Algorithm A: the complete fragment

Qed has *two* algorithms and picks between them. The first is complete on a restricted fragment; this is the paper's theoretical contribution.

**Definition 4.4** ($\mathcal{F}_T$, [2] Def. 1)**.** Fix a first-order theory $T$. The fragment $\mathcal{F}_T$ consists of `Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, where all values, projection functions and filter predicates are definable in $T$. Primary keys on the tables are allowed.

This generalises Cohen's classical UCQ result [6]: UCQ forces predicates to be conjunctions of (in)equalities and projections to be bare column references, whereas $\mathcal{F}_T$ allows *anything expressible in* $T$. With $T$ = the SMT solver's theories, that is a large class. Note $\mathcal{F}_T$ excludes `GroupBy`, `Distinct` and `Minus` — so our running example is *not* in it.

### 4.7.1 Scoped normal form (SNF)

**Definition 4.5** ([2] Def. 2)**.** $U$ is in SNF if $U = T_1 + \cdots + T_n$ where each *normal term* is

$$T = \sum_{R_1^{k_1}(s_1)} \cdots \sum_{R_m^{k_m}(s_m)} [\,P\,], \qquad \sum_{R^0(s)} U \coloneqq \sum_s \|R(s)\| \times U, \qquad \sum_{R^k(s)} U \coloneqq \sum_s \underbrace{R(s) \cdots R(s)}_{k} \times U$$

with $P$ (the *body*, $\mathrm{Bdy}\, T$) containing no table variables, and the summations ordered consistently with a fixed order on table symbols.

Auxiliary notation used below: $\mathrm{Scp}\, T$ is the list of all summation variables; $\mathrm{Scp}_R T$ those attached to table $R$; $\mathrm{Rel}\, T$ the list of table variables with their powers; $\mathrm{Rel}'T$ the same without powers.
Normalisation ([2] Alg. 1) is a straightforward induction: `Table` is already a term; `Filter` conjoins its predicate into every body; `Proj` post-composes $f$ into the body *without* adding a summation; `Join` merges scopes and conjoins bodies pairwise; `Union` concatenates terms.

### 4.7.2 Linearisation (LNF)

Two obstacles remain. First, summation variables attached to the same table can be introduced in any order, so terms that differ only by a permutation look different. Second, redundant variables: in

$$T_1 = \sum_{R(s_1)} \sum_{R(s_2)} [\,(x{=}s_2) \wedge (s_1{=}s_2)\,] \qquad \text{vs} \qquad T_2 = \sum_{R^2(s_1)} [\,x{=}s_1\,]$$

$T_1$'s $s_2$ is redundant, but the scopes have different lengths so they cannot be compared directly.

**Definition 4.6** (LNF, [2] Def. 3)**.** $U$ is in LNF if it is in SNF and, for every term $T$ and table variable $R$, $\mathrm{Bdy}\, T$ implies the variables of $\mathrm{Scp}_R T$ are pairwise distinct and form a linearly ordered chain.

Algorithm 2 achieves this by *case-splitting on the order*. For a scope $\{a, b, c\}$, Lin enumerates all 13 ways three values can be ordered-or-equal ($a{<}b{<}c$, …, $a{=}b{<}c$, …, $a{=}b{=}c$). Each term is replaced by the sum over all cases; within each case, variables proved equal are merged (the oracle decides $\mathrm{Bdy}\, T' \to s_j = s_k$), and the survivors are sorted ascending. Now every term has a canonical variable order.

*Remark* 4.7 (cost). $|\mathrm{Lin}(X)|$ is the ordered Bell number of $|X|$: $1, 1, 3, 13, 75, 541, \ldots$ Eager linearisation is therefore expensive, which is exactly why QED's *general* algorithm (§4.8) defers permutation to the matching step instead.

### 4.7.3 Unification and completeness

Terms are grouped by relational scope; within a group the summation disappears and only the bodies are compared, because

$$\sum_S [\,P_1\,] + \cdots + [\,P_n\,] \;=\; \sum_S [\,P'_1\,] + \cdots + [\,P'_m\,] \quad\Longleftarrow\quad \vec{s} \vdash_{\mathcal{O}} [\,P_1\,] + \cdots + [\,P_n\,] = [\,P'_1\,] + \cdots + [\,P'_m\,]$$

and the right-hand side is an ordinary SMT query ($[\,\cdot\,]$ becomes `ite`, $+$ becomes integer addition).

**Theorem 4.8** ([2] §4.4)**.** *On $\mathcal{F}_T$, with a complete oracle for $T$,* $[\![Q_1]\!] = [\![Q_2]\!] \Rightarrow \mathsf{unify}(\mathsf{lin}(\mathsf{snf}(Q_1)), \mathsf{lin}(\mathsf{snf}(Q_2)))$.

*Proof idea — worth understanding, it is elegant.* Contrapositive. Suppose some group of terms with relational scope $S^*$ fails its body check; choose such $S^*$ with the fewest variables. Build a database instance in which each table $R_i$ is supported on exactly the finitely many points that the failing assignment $\sigma$ touches, with multiplicity $n_i$ a free parameter. Because the term is in LNF, all variables attached to a table are distinct and ordered, so a group with scope $R_1^{k_1} \cdots R_m^{k_m}$ evaluates to the *monomial* $B_S[\delta, \sigma] \times n_1^{k_1} \cdots n_m^{k_m}$. Terms with larger scopes vanish (some variable is forced outside the support). Both sides are thus polynomials in $n_1, \ldots, n_m$, and two polynomials over $\mathbb{N}$ are equal iff all coefficients match — but the $S^*$ coefficient differs by assumption. Hence some instantiation separates $Q_1$ and $Q_2$. □

> **The idea to steal**
> LNF turns each term into a monomial whose exponents are the scope and whose coefficient is the body. Bag equivalence then becomes *polynomial identity*. That is why the paper insists on a total order on summation variables: without it the exponents are not well-defined.

## 4.8 Algorithm B: the general algorithm

For everything outside $\mathcal{F}_T$ — including our running example, which has `GroupBy` — QED uses a weaker but far more general pipeline: **normalise** → **stabilise** → **unify**. Here the oracle is allowed to answer "unknown".

### 4.8.1 SPNF ([2] Alg. 4)

Terms take the shape $\sum_{\vec{s}} [\,L\,] \times V$. The conversion is structural, and the two interesting lines are

$$\mathsf{spnf}(\|V\|) = [\,\mathsf{spnf}(V) \neq 0\,], \qquad \mathsf{spnf}(\mathrm{not}(V)) = [\,\mathsf{spnf}(V) = 0\,],$$

i.e. squash and negation are *eliminated* in favour of predicates. Products merge scopes and conjoin the conditions; sums concatenate.

### 4.8.2 Stabilisation ([2] Alg. 5)

This is QED's replacement for UDP's Lemma 3.2, and the upgrade is that it is *semantic*. For each term $\sum_{\vec{s}}[\,L\,] \times V$ and each variable $s_i$, ask whether $L$ implies $s_i = f(\text{other variables})$ for some expression $f$; if so, substitute and delete the summation.

```
function stable(U):
  U' <- 0
  for each term (Sum over s) [L] x V of U:
    s',L',V' <- s,L,V
    for each variable s_i in s:
      s'' <- s' minus s_i
      if exists f with  Gamma|Delta,s' |Phi |-_0  L' -> f(s'') = s_i :
         s' <- s'' ;  L' <- L'[f(s'')/s_i] ;  V' <- V'[f(s'')/s_i]
    U' <- U' + (Sum over s') [L'] x V'
  return U'
```

Two ways to find $f$ are given: synthesise it with SyGuS, or ask the SMT solver for the *congruence classes* of the term's subexpressions and read off an equal expression. QED implements the second for simplicity and reports the two are equally powerful in practice.

**Example 4.9** (composing two projections)**.** $\sum_{y,z}[\,x = f(y) \wedge y = g(z)\,]R(z) \stackrel{?}{=} \sum_{z}[\,x = f(g(z))\,]R(z)$. Congruence classes on the left over $\{x, y, z, f(y), g(z)\}$ are $\{x, f(y)\}, \{y, g(z)\}, \{z\}$, exposing $y = g(z)$, so $y$ is eliminated. On the right the classes are $\{x, f(g(z))\}, \{z\}$ and nothing is eliminated. The scopes now match.

*Remark* 4.10 (a real, acknowledged gap). The congruence-class method finds only expressions *already present* in the term. Given $[\,x + 3 = 10\,]$ it will not discover $x = 7$, because 7 never appears; SyGuS would. The paper reports this never mattered in the evaluation.

### 4.8.3 Unification ([2] Alg. 6)

```
function unify(U1, U2):
  drop terms whose body is provably false (oracle: not (L and squash V))
  if #terms(U1) != #terms(U2): return Unknown       # <-- ceiling of THIS algorithm
  for each term T1 of U1:
     find a term T2 of U2 with termEq(T1,T2); remove it; else return Unknown
  return True

function termEq( (Sum s1)[L1] x V1 , (Sum s2)[L2] x V2 ):
  if len(s1) != len(s2): return False
  for each bijection sigma from s2 to s1:            # capped at 24 in the
    implementation
     L2,V2 <- L2[sigma], V2[sigma]
     L1',V1',L2',V2' <- ... with every HOp/QOp replaced by a FRESH variable
     C <- congruence facts obtained by recursively unifying those operators' arguments
          under the extra assumption  Phi and (L1' or L2')
     if  Phi and C  |-_0  [L1'] x V1' = [L2'] x V2' : return True
  return False
```

Three mechanisms are packed in here; take them one at a time.

**(i) Scope permutation.** Summation is commutative, so before comparing bodies you must try bijections between the two scopes. The justification is $(\forall \vec{s}.\ f(\vec{s}) = g(\vec{s})) \Rightarrow \sum_{\vec{s}} f = \sum_{\vec{s}} g$. Note this is *pointwise* comparison: sound, but it can only ever prove sums equal by proving their summands equal. The implementation caps at 24 permutations; empirically only ∼2% of cases need more than the identity permutation.

**(ii) Higher-order operators become fresh variables.** $\mathrm{QOp}(o, \vec{v}, \lambda x.\, W)$ becomes a fresh table symbol; $\mathrm{HOp}(o, \vec{v}, Q)$ a fresh scalar. Congruence facts like $\forall x. R(x) = R'(x)$ are then *recovered* by checking that the operators match and recursively comparing their arguments.

**(iii) The ambient assumption — the genuinely new idea.** The recursive comparison of the operators' arguments is performed under the extra hypothesis $L'_1 \vee L'_2$: the conditions of the two enclosing terms. Soundness in one sentence:

**Lemma 4.11** ([2] Eq. 17)**.** *To verify* $[\,L'_1\,] \times V'_1 = [\,L'_2\,] \times V'_2$ *it suffices to check* $\Phi \wedge (L'_1 \vee L'_2) \vdash C$ *and* $\Phi \wedge C \vdash [\,L'_1\,]V'_1 = [\,L'_2\,]V'_2$.

*Proof.* If $\neg L'_1 \wedge \neg L'_2$ then $[\,L'_1\,] = [\,L'_2\,] = 0$ and both sides are 0 regardless of $C$. So assuming $L'_1 \vee L'_2$ while deriving $C$ loses nothing. □

### 4.8.4 Encoding to first-order logic

$\overline{\mathbb{N}}$-valued expressions cannot be handed to an SMT solver directly, but QED never needs their values — only whether they are zero. So $U \neq 0$ is pushed inward:

$$U_1 + U_2 \neq 0 \rightsquigarrow U_1 \neq 0 \vee U_2 \neq 0, \qquad U_1 \times U_2 \neq 0 \rightsquigarrow U_1 \neq 0 \wedge U_2 \neq 0,$$

$$\|U\| \neq 0 \rightsquigarrow U \neq 0, \qquad [\,P\,] \neq 0 \rightsquigarrow P, \qquad \sum_s U \neq 0 \rightsquigarrow \exists s.\, U \neq 0.$$

What survives is only fully-applied table symbols $R(a) \neq 0$ and operator applications, all of which are uninterpreted predicates. QED additionally assumes every table has *finite* multiplicity at each row (type $S \to \mathbb{N}$, not $S \to \overline{\mathbb{N}}$) so that no $\infty$ arises during term comparison.

> **This trick only works under a squash**
> Turning $\sum$ into $\exists$ requires that the enclosing context care about zero-vs-non-zero. When the top-level expression is *not* squashed — plain `SELECT x FROM r`, an unsquashed join — multiplicities matter and QED must fall back on pointwise term matching, which is where its incompleteness bites (§6).

*Remark* 4.12 (the two algorithms differ exactly here — do not conflate them). Alg. 6 demands a *bijection* between the two sides' terms. Alg. 3, the complete-fragment algorithm of §4.7, does **not**: it groups terms by relational scope and compares $[\,P_1\,] + \cdots + [\,P_n\,]$ with $[\,P'_1\,] + \cdots + [\,P'_m\,]$ for $n \neq m$. So "QED cannot handle differing numbers of terms" is a statement about Alg. 6 only. A `UNION ALL` split stays provable while the query remains inside $\mathcal{F}_T$; adding a `GroupBy` or `Distinct` pushes it into Alg. 6 and the bijection requirement returns.

## 4.9 The running example, proved

Translate $Q_1, Q_2$ of §2.4, with the primary key already applied:

$$Q_1(x,r) = \Big\| \sum_{y,a} [\,x{=}a\,] R(x,y)\, \|S(a)\| \Big\| \times \Big[ r = \mathrm{Sum}(\lambda y.\, \sum_a [\,x{=}a\,] R(x,y)\, \|S(a)\|) \Big]$$

$$Q_2(x,r) = \sum_a [\,x{=}a\,] \Big\| \sum_y R(x,y) \Big\| \times [\,r = \mathrm{Sum}(\lambda y.\, R(x,y))\,] \times \|S(a)\|$$

**Step 1 — stabilise.** Both terms contain $[\,x = a\,]$, which pins $a$ to $x$; congruence classes put $x$ and $a$ together, so $\sum_a$ is deleted:

$$Q_1(x, r) = \Big\| \sum_y R(x, y)\, \|S(x)\|\, [r = \mathrm{Sum}(\lambda y.\, R(x, y)\, \|S(x)\|)] \Big\|$$

$$Q_2(x, r) = \Big\| \sum_y R(x, y)\, \|S(x)\|\, [r = \mathrm{Sum}(\lambda y.\, R(x, y))] \Big\|$$

One difference remains: the $\|S(x)\|$ inside the aggregate.

*Remark* 4.13 (why $Q_2$ may be written with an outer squash). Every factor of $Q_2$ is already 0-or-1: $\big\|\sum_y R(x, y)\big\|$, the bracket, and $\|S(a)\|$. So wrapping in $\|\cdot\|$ changes nothing. But $\|S(a)\|$ is squashed *only because of the primary key* — drop the key and the factor is $S(a)$, which can be 2, and $Q_2$ is no longer squashable. That is precisely the counterexample of §2.4.4, where $Q_2$ returned $(1, 30)$ *twice*.

**Step 2 — encode.** With $v_1, v_2$ fresh for the two aggregates:

$$\phi_1(x, r) = \exists y.\, R(x, y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_1, \qquad \phi_2(x, r) = \exists y.\, R(x, y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_2$$

Goal $\forall x, r.\ \phi_1 \leftrightarrow \phi_2$; identical but for $v_1$ vs $v_2$.

**Step 3 — recursive check with the ambient assumption.** $v_1 = v_2$ needs $\lambda y.\, R(x, y)\, \|S(x)\| \equiv \lambda y.\, R(x, y)$, which is **false** standalone ($x \notin S \Rightarrow$ left is empty). Under $\phi_1 \vee \phi_2$, however, we have $S(x) \neq 0$, hence $\|S(x)\| = 1$, hence

$$(\phi_1 \vee \phi_2) \rightarrow \forall y.\ R(x, y) \times \|S(x)\| = R(x, y) \qquad \textit{valid.}$$

So $v_1 = v_2$, so $\phi_1 \leftrightarrow \phi_2$, so $Q_1 \equiv Q_2$. ■

In English: *the group-by only emits a row for an $x$ that survived the join, and any such $x$ is in $S$; so inside the aggregate $\|S(x)\|$ is just 1 and vanishes.*

## 4.10 Results and honest limits

| suite | | UDP | EQUITAS | SPES | QED′ | QED |
|---|---|---|---|---|---|---|
| Calcite (old, 232) | proved | 34 | 67 | 95 | 49 | **147** |
| Calcite (v1.32, 444) | proved | – | – | 121 | 98 | **299** |
| CockroachDB (1287) | proved | – | – | 325 | 656 | **979** |

Table 4: [2] Table 1. QED′ is the complete-fragment algorithm alone; QED is the full pipeline. Every case SPES proves, QED also proves. Average runtime 0.88 s (Calcite v1.32), of which ∼0.83 s is SMT time inside stabilisation.

| failure category | Calcite | CockroachDB |
|---|---|---|
| aggregates | **96** | 27 |
| custom operators | 25 | **152** |
| list semantics (`ORDER BY`/`LIMIT`) | 18 | 103 |
| typing (implicit casts) | 6 | 26 |

Table 5: [2] Table 3. Aggregates dominate on Calcite — the direct cost of leaving $\alpha$ uninterpreted. SQLSolver attacks exactly this column.

# 5 U-expressions + LIA*: SQLSolver

*Source: Ding, Wang, Yang, Zhang, Xu, Chen, Piskac, Li. PACMMOD 1(4), Art. 227 (SIGMOD), 2023 [3].*

## 5.1 The diagnosis

SQLSolver keeps the U-expression formalism — it adds new *translation* rules (§5.5) but no new algebraic operators — and asks a sharper question: *what, specifically, blocks a semantics-based prover?* Its answer is the unbounded summation.

Once you translate two queries to $f_1(t)$ and $f_2(t)$, equivalence is the unsatisfiability of $\exists t.\ f_1(t) \neq f_2(t)$. If $f_1, f_2$ were plain arithmetic an SMT solver would settle it. But they contain $\sum_t$, and no SMT solver has a theory of unbounded sums. Prior systems coped by:

- UDP: demand the summations be *syntactically identical* after normalisation.
- WeTune: a handful of heuristics — essentially only the case where both sides have exactly one summation, handled by comparing the summands.

Neither survives contact with real queries. The paper quantifies it: on the Calcite suite, 180 of the pairs have unbounded summations in their U-expressions; UDP handles 33, WeTune 71.

**Example 5.1** (the motivating failure — differing *numbers* of summations)**.**

```
    Q1: select x from R where y<=1000
    union all
    select x from R where y>1000 and z<500
Q2: select x from R where y<=1000 or z<500
```

$$f_1(t) = \sum_{t_1} E_1 + \sum_{t_1} E_2, \qquad f_2(t) = \sum_{t_1} E_3$$

with $E_1 = [\,t{=}x(t_1)\,]R(t_1)[\,y(t_1){\leq}1000\,]$, $E_2 = [\,t{=}x(t_1)\,]R(t_1)[\,y(t_1){>}1000\,][\,z(t_1){<}500\,]$, $E_3 = [\,t{=}x(t_1)\,]R(t_1)[\,y(t_1){\leq}1000 \vee z(t_1){<}500\,]$.

**Two summations on the left, one on the right.** UDP rejects immediately (its Alg. 2 returns false when the term counts differ). WeTune's rules do not cover $\sum + \sum$. Yet the queries are obviously equivalent, since the two filters are disjoint and cover the disjunction.

*Fairness note.* Qed — published after SQLSolver — *does* handle this shape, because the query lies in its complete fragment $\mathcal{F}_T$ and Alg. 3 groups terms by relational scope, comparing $[\,P_1\,] + [\,P_2\,]$ with $[\,P_3\,]$ pointwise. What LIA* adds is generality: it requires neither a shared relational scope nor membership in a fragment, and it keeps working under aggregates, nesting and non-linearity (§5.4).

## 5.2 LIA and LIA*

**Definition 5.2** (LIA)**.** Linear integer arithmetic: first-order formulas over $\leq$ and $=$ whose terms are $T ::= x \mid c \mid T_1 + T_2 \mid c \times T_1 \mid \mathrm{ite}(F, T_1, T_2)$. Note $c \times T$ — multiplication by a *constant* only. Two variables may never be multiplied. Decidable, and Z3 is good at it.

**Definition 5.3** (additive closure)**.** For a set $S$ of integer vectors, $S^* = \{\, \vec{v} \mid \vec{v} = \sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \,\}$ — all non-negative integer combinations of members of $S$, with $n$ arbitrary.

**Definition 5.4** (LIA*)**.** A formula of the shape $\exists \vec{u}, \vec{v}.\ F_1(\vec{u}, \vec{v}) \wedge \vec{v} \in \{\vec{x} \mid F_2(\vec{x})\}^*$ with $F_1, F_2$ in LIA.

**Why $\{\cdot\}^*$ is exactly the right tool**
$\sum_t f(t)$ adds up $f$ over an unknown, unbounded number of tuples. $S^*$ is the set of things obtainable by adding up an unknown, unbounded number of members of $S$. Take $S$ to be the set of possible *per-tuple contribution vectors*, and the value of the summation is precisely an element of $S^*$. The unboundedness moves from the sum's index into the closure operator, where a decision procedure already exists.

**Theorem 5.5** ([7, 8]; [3] Thm. 4.1)**.** *Every LIA* formula has an equisatisfiable LIA formula: there is a $k$ with*

$$\exists \vec{u}, \vec{v}, \vec{x}_1, \ldots, \vec{x}_k, \lambda_1, \ldots, \lambda_k.\ F_1(\vec{u}, \vec{v}) \wedge \vec{v} = \sum_{i=1}^{k} \lambda_i \vec{x}_i \wedge F_2(\vec{x}_1) \wedge \cdots \wedge F_2(\vec{x}_k).$$

*Satisfiability of LIA* is NP-complete.*

The $\vec{x}_i$ are *generators* of $F_2$: every solution of $F_2$ is a non-negative combination of them. SQLSolver computes them with the incremental approximation method of Levatich et al. [8].

**Example 5.6** (generators, concretely)**.** For $F_2 \equiv (x_3 = x_1 + x_2)$ the generators are $(1,0,1)$ and $(0,1,1)$: every solution is $\lambda_1(1,0,1) + \lambda_2(0,1,1)$. So $\vec{v} \in \{\vec{x} \mid x_3 = x_1 + x_2\}^*$ becomes $\exists \lambda_1, \lambda_2.\ \vec{v} = \lambda_1(1,0,1) + \lambda_2(0,1,1)$, which is plain LIA.

### 5.3 The algorithm, on Example 5.1

**Step 1.** Translate to U-expressions; the goal is unsatisfiability of $\exists t.\ f_1(t) \neq f_2(t)$, i.e. $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$.

**Step 2.** Name each summation. Introduce $v_i$ with $v_i = \sum_{t_1} E_i$ and write them as one vector equation:

$$\exists t, v_1, v_2, v_3.\ v_1 + v_2 \neq v_3 \ \wedge\ (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3).$$

*This vectorisation is the key move*: three separate scalar sums over the same index become one vector-valued sum, which is what the closure operator consumes.

**Step 3.** Replace the vector sum by additive closure:

$$\exists t, \vec{v}.\ v_1 + v_2 \neq v_3 \ \wedge\ \vec{v} \in \{(x_1, x_2, x_3) \mid x_1{=}E_1 \wedge x_2{=}E_2 \wedge x_3{=}E_3\}^*.$$

**Step 4.** Replace U-expression syntax by LIA terms: $[b] \mapsto \mathrm{ite}(b, 1, 0)$, $\|E\| \mapsto \mathrm{ite}(E{=}0, 0, 1)$, $\mathrm{not}(E) \mapsto \mathrm{ite}(E{=}0, 1, 0)$, and every tuple variable, attribute access $x(t_1)$ and table application $R(t_1)$ becomes a fresh integer variable. With $x_4 \coloneqq t$, $x_5 \coloneqq x(t_1)$, $x_6 \coloneqq R(t_1)$, $x_7 \coloneqq y(t_1)$, $x_8 \coloneqq z(t_1)$:

$$\begin{aligned} x_1 &= \mathrm{ite}(x_4{=}x_5 \wedge x_7{\leq}1000,\ x_6,\ 0) \\ x_2 &= \mathrm{ite}(x_4{=}x_5 \wedge x_7{>}1000 \wedge x_8{<}500,\ x_6,\ 0) \\ x_3 &= \mathrm{ite}(x_4{=}x_5 \wedge (x_7{\leq}1000 \vee x_8{<}500),\ x_6,\ 0) \end{aligned}$$

**Step 5.** Compute generators and emit LIA. Here the solver infers $x_1 + x_2 = x_3$ from the `ite` definitions, so the generators are $(1,0,1)$ and $(0,1,1)$, giving

$$\exists \vec{v}, \lambda_1, \lambda_2.\ v_1 + v_2 \neq v_3 \wedge \vec{v} = \lambda_1(1,0,1) + \lambda_2(0,1,1),$$

i.e. $\vec{v} = (\lambda_1, \lambda_2, \lambda_1{+}\lambda_2)$, which forces $v_3 = v_1 + v_2$ and contradicts $v_1 + v_2 \neq v_3$. **Unsatisfiable ⇒ the queries are equivalent.** ■

**What just happened, in one sentence**

SQLSolver never matched the two sides structurally. It reasoned *arithmetically* about the relationship between three sums that share an index — and the relationship $x_1 + x_2 = x_3$ holds pointwise, so it lifts to the sums. Structural provers cannot express that step at all.

## 5.4 Three complications the textbook theory does not cover

Real U-expressions violate the shape LIA* expects in three ways. Each got a new procedure.

**(1) Nested summations.** A summation whose body contains another summation — which is what subqueries, `EXISTS` and aggregates produce. SQLSolver replaces each inner $\sum e_j$ with a fresh $y_j$ and applies the closure operator again, producing a *nested* LIA* formula $\ell := \exists \vec{u}, \vec{v}.\ F(\vec{u}, \vec{v}) \wedge \vec{v} \in \{\vec{x} \mid \ell'(\vec{x})\}^*$.

**Theorem 5.7** ([3] Thm. 4.2)**.** *If $\ell'(\vec{x})$ is equivalent to a LIA formula $F_2$, then $\ell$ is equivalent to $\exists \vec{u}, \vec{v}.\ F_1(\vec{u}, \vec{v}) \wedge \vec{v} \in \{\vec{x} \mid F_2(\vec{x})\}^*$.*

So nesting is handled by *recursion*: flatten the innermost LIA* to LIA, then the next one out, and so on.

**(2) Parameterised summations.** A summation whose body has free variables bound outside it, e.g. $t_1$ in $\sum_{t_1}([\,a(t_1){>}3\,] \times \sum_{t_2}[\,a(t_1){=}a(t_2)\,]R(t_2))$; also any summation mentioning non-integer values (strings, reals), which LIA* has no sorts for.

**(3) Non-linear summations.** $\sum_{t_1,t_2} R(t_1) \times S(t_2)$ becomes $\sum(x_1 \times x_2)$ — a product of two variables, forbidden in LIA. Joins and `INTERSECT` produce these constantly. The fix is **over-approximation**: replace the product $x_1 \times x_2$ by a fresh variable $y$, constrained only by $y = 0 \leftrightarrow (x_1 = 0 \vee x_2 = 0)$.

**Definition 5.8** (over-approximation)**.** $Q$ over-approximates $P$ if $\neg Q \to \neg P$. Equivalently every model of $P$ is a model of $Q$.

**The soundness direction, and what it costs**

SQLSolver builds an over-approximation $Q$ of $\exists t. f_1(t) \neq f_2(t)$. If $Q$ is **unsat** then the original is unsat, so the queries are equivalent — the answer you want is sound. If $Q$ is **sat** you learn *nothing*: the queries may still be equivalent. The paper states this plainly as reason (2) for incompleteness. So SQLSolver is a prover, never a refuter — a "sat" result is not a counterexample.

## 5.5 Extended U-expressions: modelling more SQL

SQLSolver's second contribution is a larger translation table [3, Tab. 3–4].

| | |
|---|---|
| `INTERSECT` | $\|f_l(t) \times f_r(t)\|$ |
| `INTERSECT ALL` | $E \times f_l(t) + \text{not}(E) \times f_r(t)$ where $E = [\,f_l(t) \le f_r(t)\,]$ (i.e. the pointwise *minimum*) |
| `VALUES (t1),...` | $[\,t{=}t_1\,] + \cdots$ |
| scalar subquery | $[\,1{=}\sum_{t_1} E\,] \times \|\sum_{t_1}(E \times [\,t{=}a(t_1)\,])\| + \mathit{IsNull}(t) \times \text{not}(\sum_{t_1} E)$ |
| `CASE WHEN` | $[\,p_0\,][\,a{=}a_0\,] + \cdots + \text{not}([\,p_0\,]) \cdots \text{not}([\,p_{n-1}\,])[\,a{=}a_n\,]$ |
| `a IN (v1,...)` | $\|[\,a(t){=}v_1\,] + \cdots\|$ |

The scalar-subquery row repays study: it says "if the subquery returns exactly one row, $t$ is that row's value; if it returns none, $t$ is Null". The $> 1$ row case is a runtime error and is simply not modelled.

**Concrete aggregates — the big one.** UDP, WeTune and Qed all treat aggregates as uninterpreted. SQLSolver writes them out. Every entry has the same skeleton:

$$f(t_l \cdot t_r) \coloneqq \underbrace{[\,p(t_r)\,]}_{\text{HAVING}} \times \underbrace{\Big\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,] \Big\|}_{\text{the group is non-empty}} \times \underbrace{(\text{characterisation of } t_r)}_{\text{the aggregate itself}}$$

where the output row is the concatenation $t_l \cdot t_r$ of group key and aggregate value. The third factor per function:

$$\begin{aligned}
\textsc{Count}: &\quad \Big[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}(y(t_1))\,])\Big] \\
\textsc{Sum}: &\quad \Big[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}(y(t_1))\,])\,y(t_1)\Big] \\
\textsc{Count Distinct}: &\quad \Big[t_r = \sum_{t_1} \Big\| \sum_{t_2} R(t_2)[\,t_l{=}x(t_2)\,][\,t_1{=}y(t_2)\,]\mathrm{not}([\,\mathit{IsNull}(y(t_2))\,]) \Big\|\Big] \\
\textsc{Max}: &\quad \mathrm{not}\Big(\sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,][\,y(t_1){>}t_r\,]\Big) \times \Big\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,][\,y(t_1){=}t_r\,] \Big\| \\
\textsc{Avg}: &\quad [t_r \times (\textsc{Count expr}) = (\textsc{Sum expr})]
\end{aligned}$$

Three techniques worth noting. Sum is literally $\sum_y y \times (\text{multiplicity})$ — the definition from §2 written in the algebra. Max is not computed but *characterised* by two properties ("no element exceeds $t_r$" and "some element equals $t_r$"), because a maximum cannot be expressed as a sum. Avg avoids division by asserting $t_r \times \text{count} = \text{sum}$.

*Remark* 5.9 (the payoff). This is what lets SQLSolver prove `select a, count(b) from R group by a` ≡ `select S.a, sum(S.cnt) from (select a, count(b) as cnt from R group by a) S group by S.a` — a sum of counts equalling one count. No uninterpreted-aggregate prover can do this, at any level of cleverness, because the fact needed is a fact *about* Sum and Count.

### 5.6 Ordered bag semantics (`ORDER BY`)

U-expressions model bags, which have no order, so `ORDER BY` is out of reach by construction. SQLSolver adds a separate, syntactic layer.

**Definition 5.10** ([3] Def. 1)**.** If $Q_1$'s result is ordered, $Q_2 \equiv Q_1$ iff (i) they contain the same multiset of tuples *and* (ii) any two tuples appear in the same relative order in both.

**Step 1 — simplify.** Delete `ORDER BY` in a subquery unless followed by `LIMIT`/`OFFSET` (it has no observable effect); replace `... ORDER BY e LIMIT 0` by the empty table; hoist an `ORDER BY` out of the left arm of a `UNION ALL`/outer join when the same clause is applied outside; and merge stacked `ORDER BY ... LIMIT n1 OFFSET o1` / `LIMIT n2 OFFSET o2` into one with $o_3 = o_1 + o_2$ and $n_3 = n_2$ if $o_2 + n_2 \le n_1$, else $\max(n_1 - o_2, 0)$.

**Step 2 — divide and conquer.** Pick an `ORDER BY` in $Q_1$; find the matching ones in $Q_2$ (same sort attributes, same `LIMIT`/`OFFSET`). For each candidate, recursively prove the two *sorted subqueries* $Q_3 \equiv Q_4$ with the bag algorithm, then replace both by the same fresh relation and recurse on what remains.

**Theorem 5.11** ([3] Thm. 4.4)**.** *If the check of Step 2 passes after simplification, the queries are equivalent under ordered bag semantics.*

*Remark* 5.12. This layer is orthogonal: the paper combines it with UDP, SPES and WeTune too. Of 400 pairs, 39 needed it. It is also acknowledged as a source of incompleteness — it is syntactic, and cannot handle every `ORDER BY` query.

### 5.7 Guarantees

**Theorem 5.13** ([3] Thm. 5.1)**.** *SQLSolver is sound under both bag and ordered bag semantics.*

Incompleteness has three named sources [3, §5]: (1) unmodelled SQL (lateral subqueries, window functions); (2) the over-approximation used for non-linear summations, which is not equisatisfiable; (3) the syntactic `ORDER BY` layer. The paper reports that (2) and (3) caused no false negatives in its evaluation — all failures were unmodelled features.

**Theorem 5.14** ([3] Thm. 5.2)**.** *SQLSolver is complete for CQ and UCQ, and additionally for queries of the form* $Q'_1$ `UNION ALL` $\cdots$ `UNION ALL` $Q'_n$ *where each* $Q'_i$ *is* `SELECT R.a1..aj, agg1(R.b1)..aggk(R.bk) FROM Ri WHERE p GROUP BY .. HAVING q` *or* `SELECT [DISTINCT] R.a1.. aj FROM Ri WHERE p`*, with aggregates restricted to Count, Max, Min, and predicates decidable by SMT.*

This second fragment is the interesting one: it is *disjoint* from what UCQ-complete provers cover, since it contains group-by and aggregation.

### 5.8 Results

| suite | UDP | SPES | WeTune | SQLSolver |
|---|---|---|---|---|
| Calcite (232) | 33 | 99 | 78 | **232** |
| Spark SQL (127) | 20 | 56 | 31 | **114** |
| TPC-C (19) | – | 15 | – | **19** |
| TPC-H (22) | – | 0 | – | **19** |

Table 6: [3] Tables 1 and §6.2. Of 400 pairs, the three prior provers prove 207 between them; SQLSolver proves 384, a superset. 16 pairs are proved by nobody.

Failure attribution among the pairs SQLSolver proves [3, Tab. 5]: UDP misses 132 for unsupported features, 160 for its checking algorithm, 39 for ordering; SPES 103/72/39; WeTune 63/181/31. Read the middle column as "the cost of not having $\mathsf{LIA}^*$".

### 5.9 SQLSolver on the running example

SQLSolver reports proving all 232 of its Calcite pairs, and the aggregate-pushdown rule is a Calcite rule, so it is very likely covered — but we cannot verify *which* pairs those are from the paper alone, so we describe the mechanisms rather than assert a result.

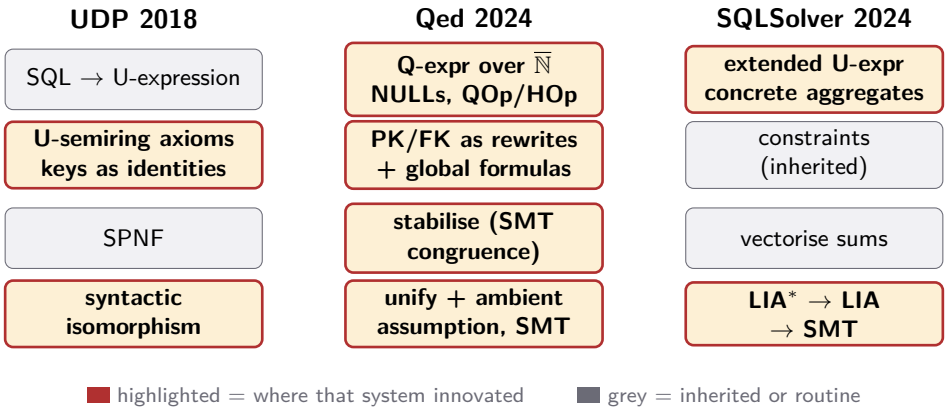
Two of SQLSolver's three strengths bear on it. Concrete Sum means the two aggregates are related by a real fact, not just congruence. $\mathsf{LIA}^*$ means differing summation structure is not fatal. The third element — the primary key — is *not* a SQLSolver contribution: constraint handling is inherited from UDP/WeTune, and the paper's own related-work section says so. On this particular rewrite the key is the crux, so SQLSolver's advantage over Qed is smaller here than its headline numbers suggest.

| | |
|---|---|
| **Strong** | a principled decision procedure for unbounded sums; differing numbers of summations and of aggregates are no obstacle; concrete SUM/COUNT/MAX/MIN/AVG; `INTERSECT ALL`, `VALUES`, scalar subqueries, `CASE`; `ORDER BY` via a separate layer; complete for a group-by fragment no one else covers. |
| **Weak** | NULL handling inherited from WETUNE, not developed; integrity constraints inherited from UDP/WETUNE; no first-class story for unknown operators; the non-linear path is a one-way over-approximation; lateral subqueries and window functions unmodelled; the concrete aggregate tables are a large hand-written trusted base. |

### 5.10 Scorecard

## 6 The three approaches compared

### 6.1 Where the effort went




The picture says it: QED rebuilt the *modelling* and the *search*; SQLSOLVER rebuilt the *translation of aggregates* and the *arithmetic backend*. They are close to orthogonal.

### 6.2 Capability matrix

| | UDP | QED | SQLSOLVER |
|---|---|---|---|
| semantic domain | any U-semiring | $\overline{\mathbb{N}}$ only | any U-semiring |
| predicate reasoning | congruence closure | SMT | SMT |
| NULL / 3-valued logic | no | full | partial (from WETUNE) |
| outer joins | no | yes | partial |
| primary / foreign keys | identities | rewrite + global formula | inherited |
| FK cycles terminate | no | yes (lazy) | – |
| aggregates | uninterpreted | uninterpreted (HOp) | concrete |
| unknown operators | unsupported | QOp/HOp first class | unsupported |
| differing #summations | no | only inside $\mathcal{F}_T$ | yes (LIA*) |
| `ORDER BY` | no | no | yes (separate layer) |
| bag `INTERSECT ALL` | no | no (needs monus) | yes |
| completeness fragment | UCQ (set *or* bag) | $\mathcal{F}_T$ (UCQ + any theory $T$) | CQ/UCQ + a group-by fragment |
| refutation | no | no | no |

### 6.3 A gallery of separating examples

The fastest way to understand three provers is to see what each one alone can do.

**(1) Syntactically different but equal predicates.** `WHERE a+b = 0` versus `WHERE a = -b`. UDP fails — congruence closure treats + as uninterpreted. QED and SQLSOLVER succeed — both call an SMT solver on predicates. This single limitation is why *everyone* after 2018 uses SMT.

**(2) Aggregate pushdown through a key join** (§2.4). Needs: primary key as an algebraic fact *and* context-sensitive comparison of the two aggregate arguments. QED succeeds via $S \rightsquigarrow \lambda a.\ \|S(a)\|$ plus the $\phi_1 \vee \phi_2$ ambient assumption. UDP fails on the context-free recursive check (§3.10). SPES fails — no applicable key rule.

**(3) Differing numbers of summations** (Example 5.1). SQLSOLVER succeeds via LIA*. UDP fails at Alg. 2 line 3 (term counts differ). QED also succeeds here — but only because this query lies in $\mathcal{F}_T$, so the *complete-fragment* algorithm applies: Alg. 3 groups terms by relational scope and compares $[P_1] + \cdots + [P_n]$ against $[P'_1] + \cdots + [P'_m]$ with $n \neq m$ allowed. This exact pattern is the paper's own motivating example for that step. *Outside* $\mathcal{F}_T$ — i.e. as soon as a `GroupBy`, `Distinct`, `Minus` or unknown operator appears — QED falls back to Alg. 6, which needs a **bijection** between terms and returns UNKNOWN when the counts differ. And neither QED algorithm relates sums with *different relational scopes*; LIA* does, and it applies uniformly rather than only inside a fragment.

```
Q1: select a, count(b) from R group by a
Q2: select S.a, sum(S.cnt)
    from (select a, count(b) as cnt from R group by a) S group by S.a
```

SQLSOLVER succeeds — it knows what SUM and COUNT mean. UDP and QED cannot, at any level of cleverness: with aggregates uninterpreted, the required fact is simply not expressible. This is why "aggregates" is QED's largest Calcite failure bucket (96 cases).

**(5) Redundant self-join on a unique key** ([2] §6.3.2).

```
Q1: select R.a from R join S S1 on R.a=S1.k join S S2 on R.a=S2.k
Q2: select R.a from R join S on R.a=S.k
```

QED succeeds: stabilisation finds $k_1 = k_2 = a$ through the SMT congruence classes, then $\|S(a)\| \times \|S(a)\| = \|S(a)\|$. SPES fails: its self-join-on-key rule matches $R \bowtie (S \bowtie S)$, not $(R \bowtie S) \bowtie S$, and it cannot infer $S_1.k = S_2.k$ transitively. **Note this example has no top-level squash** — the answer genuinely has multiplicity 2 — so QED discharges it by proving the summands equal *pointwise*, not by turning $\sum$ into $\exists$.

**(6) Predicate pushdown through a left join** ([2] §6.3.1).

```
Q1: select * from (select * from R where R.a=42) X left join S on X.a=S.b
Q2: select * from (select * from R where R.a=42)
                  left join (select * from S where S.b=42) on TRUE
```

QED succeeds. SPES fails, and QED's paper reports the reason as a soundness bug: it states that one of SPES's normalisation rules *unsoundly* rewrites a left join whose condition is `TRUE` into a cross join. (We report this as [2]'s claim; we did not reproduce it.) Anything needing NULL semantics is out of reach for UDP entirely.

**(7) `ORDER BY` anything.** SQLSOLVER only.

### 6.4 The deep structural difference: squashed versus not

Everything above has one root cause, and it is worth isolating because it predicts which examples a given prover will handle.

When the top-level expression is **squashed** — `SELECT DISTINCT`, `GROUP BY`, `EXISTS` — multiplicities are irrelevant. QED can then push $\neq 0$ inward, turning $\sum$ into $\exists$, $\times$ into $\wedge$ and $+$ into $\vee$, and hand a pure Boolean formula to SMT. Cheap and strong.

When it is **not squashed** — plain projection, a join, a `UNION ALL` — the counts *are* the answer, and the two systems fall back on comparing summands pointwise, justified by $(\forall \vec{s}.\ f(\vec{s}) = g(\vec{s})) \Rightarrow \sum_{\vec{s}} f = \sum_{\vec{s}} g$. That is sound but demands *alignable scopes*: UDP's Alg. 2 and QED's Alg. 6 both require a one-to-one match between terms. (QED's Alg. 3 relaxes this to "same relational scope", which is strictly weaker but only available inside $\mathcal{F}_T$.) SQLSOLVER instead reasons about the sums' *values*, which needs no alignment at all.

**Why the squash matters in practice**

Under a squash the comparison is Boolean, so a `UNION ALL` split like Example 5.1 reduces to $\exists y.(y{\leq}1000 \wedge R{\neq}0) \vee \exists y.(y{>}1000 \wedge z{<}500 \wedge R{\neq}0) \leftrightarrow \exists y.(\ldots)$ — an ordinary SMT query that any SMT-backed tool settles. Without the squash the same rewrite is an *arithmetic* statement about sums. QED can still discharge it *if* the terms share a relational scope and the query is in $\mathcal{F}_T$ (Alg. 3 sums the bodies); otherwise it needs a term bijection. LIA* needs neither condition.

### 6.5 On the published numbers

| reported by | benchmark described as | SPES | the paper's own tool |
|---|---|---|---|
| QED [2] | "232 Calcite pairs" | **95** | 147 |
| SQLSOLVER [3] | "232 Calcite pairs" | **99** | 232 |

The same baseline on nominally the same benchmark differs. The extraction pipelines are different, so the suites are different, so the percentages are not comparable. Additional cautions:

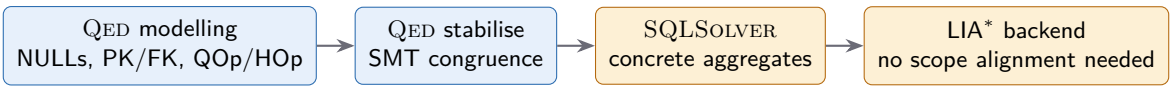
- QED's harder suites (444 non-trivial Calcite v1.32 pairs; 1287 CockroachDB pairs) were never run through SQLSOLVER, and vice versa.
- QED does not evaluate against SQLSOLVER at all — the two are near-simultaneous.
- SQLSOLVER's TPC-C/TPC-H pairs were generated by running Spark SQL's optimiser and then *manually* converting logical plans back into SQL. That is a reasonable methodology but it is not an independent benchmark.
- "Proves 100% of Calcite" should be read as "100% of the pairs this group extracted and chose to model".

**If you are writing a paper in this area**

Do not cite a cross-paper number as a head-to-head comparison. The only defensible comparison is one you run yourself, on one extraction, with all tools on the same input. As of these two papers, nobody has done that.

### 6.6 What the combination would look like

The two 2024 systems are complementary to an unusual degree, and the merge is not speculative — each piece has a clean interface.




QED's term-bijection step is its ceiling; LIA* is exactly the tool that removes that ceiling. QED's uninterpreted HOp for aggregates is its largest failure bucket; SQLSOLVER's aggregate tables fill it. Conversely SQLSOLVER's weakest areas — NULLs, constraints, unknown operators — are precisely QED's contributions. No published system does this.

## 7 An implementer's guide

This section is the advice we wish the papers had included. It assumes you are building on Calcite (all of SPES, QED and SQLSOLVER consume relational plans, and Calcite's `RelNode` is the easiest source).

### 7.1 Build order

Do not start with the fancy parts. In dependency order:

1. **Plan → algebra.** Handle `TableScan`, `Filter`, `Project`, `Join`, `Union(all=true)` only. Represent an expression as a sum of terms; a term as (scope, body predicate, list of table applications).
2. **Normalisation to sum-of-products.** Pure syntax; no solver needed. Test by round-tripping and by evaluating both forms on small random instances.
3. **Pointwise term matching** with scope permutation. At this point you can already prove filter/projection/join reordering.
4. **Bring in SMT for predicates.** This is the single biggest capability jump per line of code — it is the whole difference between UDP and its successors.
5. **Stabilisation** via congruence classes. Second biggest jump.
6. **Primary keys.** One rewrite. Suddenly a large class of real rules becomes provable.
7. **Squash/`Distinct`/`GroupBy`**, then the $\neq 0$ Boolean encoding.
8. Only then: NULLs, foreign keys, uninterpreted operators, LIA*.

### 7.2 Data structures

```
Term   = { scope : List<(TableSym, Var, int power)>,   # Sum over R^k(s)
           body  : Formula,                            # the [P] part, SMT-encodable
           value : Expr }                              # the V part (QOp/HOp
    applications)
Expr   = List<Term>                                    # a sum of terms
```

Keep `body` in your SMT library's AST from the very beginning. Converting a home-grown predicate type into SMT later is a rewrite you will do twice.

### 7.3 Ten traps, each of which will cost you a day

1. `GROUP BY` **needs a squash.** UDP's printed de-sugaring (Remark 3.4) iterates over every row of $R$, not every distinct key. Use QED's two-factor rule: $\|\sum_s [\, x = k(s)\,] Q(s)\| \times [\, y = \alpha(\ldots)\,]$. Get this wrong and multiplicities are silently wrong for duplicated keys.

2. **The primary-key rule for tables with non-key columns.** Eq. (21), $R \rightsquigarrow \lambda k.\ \|R(k)\|$, applies only when the key is the *whole* schema. In general use Eq. (20) with the functional-dependency symbol $f_R$. Almost every write-up quotes the easy form.

3. **Foreign-key chasing may not terminate.** UDP says so explicitly: a cycle in the FK graph loops. Either bound the depth or do what QED does — emit $\forall b, k.\ \|S(b,k)\| \to \|R'(k)\|$ as one global assertion and let the solver unfold it lazily.

4. **Over-approximation is one-directional.** In SQLSOLVER's non-linear path, UNSAT means equivalent, SAT means *nothing*. If you report "not equivalent" on SAT you have an unsound tool.

5. **Permutation blowup.** termEq enumerates bijections between scopes; QED caps at 24 and reports ∼98% of cases match on the identity permutation. Try identity first, always.

6. **Linearisation blowup.** $\mathrm{Lin}(X)$ is the ordered Bell number: $1, 1, 3, 13, 75, 541, 4683$. Eager linearisation on a term with five variables already generates 541 cases *per table*. Use it only inside the complete fragment.

7. **Congruence classes only find expressions already present.** Given $[\, x + 3 = 10\,]$ they will not produce $x = 7$. If you need that, call SyGuS.

8. **Finiteness.** QED assumes every table has finite multiplicity at each row so that $\infty$ never appears during term comparison. If you model tables as $S \to \overline{\mathbb{N}}$ you must handle $\infty$ arithmetic; if as $S \to \mathbb{N}$ you must not claim completeness over all U-semirings.

9. **Bag** `EXCEPT ALL` / `INTERSECT ALL` **are not free.** They need truncated subtraction (monus). QED declines and treats them as unknown operators; SQLSOLVER encodes `INTERSECT ALL` as $[\, f_l \le f_r\,] f_l + \mathrm{not}([\, f_l \le f_r\,]) f_r$, the pointwise minimum. Pick one deliberately.

10. **Implicit casts.** QED's failure table lists "typing" for 6 Calcite and 26 CockroachDB pairs. Calcite inserts `CAST` nodes freely; if you model casts as uninterpreted functions, two plans that differ only in an implicit cast will not unify. Decide early whether to erase casts (unsound in general) or model them.

### 7.4 Testing: use a real database

The cheapest high-value test harness is differential testing against PostgreSQL, and it catches translation bugs that no amount of staring will.

```
for each candidate rewrite (Q1, Q2):
    1. generate small random instances (0-5 rows, heavy duplicates, some NULLs)
    2. run both queries in Postgres, compare as BAGS:
         (Q1 EXCEPT ALL Q2) must be empty  AND  (Q2 EXCEPT ALL Q1) must be empty
    3. if they differ  -> your prover must NOT say "equivalent"
    4. also evaluate your U-/Q-expression directly on the same instance
       and check it reproduces Postgres' multiplicities row by row
```

Step 4 is the one people skip and the one that matters: it validates the *translation*, independently of the decision procedure. Two specific settings to include:

- **duplicates in the right-hand table of every join** — this is what separates a correct key model from a wrong one. Our running example produces $(1, 60)$ versus two copies of $(1, 30)$ (§2.4.4); if your translation does not reproduce exactly that, your `GroupBy` or key rule is wrong.
- `NULL` **in every nullable column** — three-valued `AND`/`OR` is easy to get subtly wrong, and `Null And False` = `False` is the canonical check.

Use a session-local table to shadow a real one when you want a no-key variant, so you never mutate the reference schema:

```
create temp table s (a int, b int);          -- hides the real s, no primary key
insert into s values (1,100),(1,100);
```

### 7.5 Getting plans out of Calcite

For the record, this is all it takes to obtain the trees in §2.4:

```
SchemaPlus root = Frameworks.createRootSchema(true);
root.add("R", tableOf("X","Y", /*rows*/1e6, /*key*/null));
root.add("S", tableOf("A","B", /*rows*/5e4, /*key*/0));   // column 0 unique
FrameworkConfig cfg = Frameworks.newConfigBuilder().defaultSchema(root).build();
Planner p = Frameworks.getPlanner(cfg);
RelNode rel = p.rel(p.validate(p.parse(sql))).rel;
System.out.println(RelOptUtil.toString(rel, SqlExplainLevel.EXPPLAN_ATTRIBUTES));
```

Declare uniqueness through the table's `Statistic`: `Statistics.of(rows, ImmutableList.of(ImmutableBitSet.of(0)))`. That one bit is what makes Calcite's `AggregateJoinTransposeRule` produce $Q_2$ instead of the compensated `SUM` $\times$ `COUNT` plan of §2.4.2 — and it is the same bit your prover needs for the key rewrite. A `Planner` instance is single-use; build a fresh one per query.

## A All axioms and definitions on one page

### U-semiring (UDP) — the complete axiom set

$(\mathcal{U}, 0, 1, +, \times)$ is a commutative semiring: associativity and commutativity of $+$ and $\times$; $0$ is the unit of $+$; $1$ of $\times$; $\times$ distributes over $+$; $0 \times x = 0$.

| **squash** | **negation** |
|---|---|
| $\|0\| = 0 \quad \|1 + x\| = 1$ | $\text{not}(0) = 1$ |
| $\|\|x\| + y\| = \|x + y\|$ | $\text{not}(x \times y) = \|\text{not}(x) + \text{not}(y)\|$ |
| $\|x\| \times \|y\| = \|x \times y\|$ | $\text{not}(x + y) = \text{not}(x) \times \text{not}(y)$ |
| $\|x\| \times \|x\| = \|x\|$ | $\text{not}(\|x\|) = \|\text{not}(x)\| = \text{not}(x)$ |
| $x \times \|x\| = x$ | |
| $x^2 = x \;\Rightarrow\; \|x\| = x$ | |
| **unbounded summation** | **predicates & equality** |
| $\sum_t (f_1 + f_2) = \sum_t f_1 + \sum_t f_2$ | $[b] = \|[b]\|$ |
| $\sum_{t_1} \sum_{t_2} f = \sum_{t_2} \sum_{t_1} f$ | $[e_1{=}e_2] + [e_1{\neq}e_2] = 1$ |
| $x \times \sum_t f = \sum_t (x \times f)$ | $f(e_1)[e_1{=}e_2] = f(e_2)[e_1{=}e_2]$ |
| $\|\sum_t f(t)\| = \|\sum_t \|f(t)\|\|$ | $\sum_t [t{=}e] = 1$ |
| **derived:** $\sum_t [t{=}e] \times f(t) = f(e)$ | **(Lemma 3.2 — the workhorse)** |
| **KEY** $R.k$: $[t.k{=}t'.k] \times R(t) \times R(t') = [t{=}t'] \times R(t)$ | |
| **FOREIGN KEY** $S.k' \to R.k$: $S(t') = S(t') \times \sum_t R(t) \times [t.k{=}t'.k']$ | |

### Q-expressions (Qed) — definitions, not axioms

Domain $\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}$ with $\infty + a = \infty$, $0 \times \infty = 0$, $a \times \infty = \infty$ for $a \neq 0$.

$$\sum_s f(s) = \begin{cases} f(s_1) + \cdots + f(s_n) & \text{supp}(f) = \{s_1..s_n\} \text{ finite} \\ \infty & \text{else} \end{cases} \qquad [\,P\,] = \begin{cases} 1 & P \\ 0 & \neg P \end{cases} \qquad \|a\| = [\,a \neq 0\,] \qquad \text{not}(a) =$$

| | |
|---|---|
| **primary key** (general) | $R \rightsquigarrow \lambda k, s.\ \|R'(k)\| \times [\,s = f_R(k)\,]$ |
| **primary key** (key = whole schema) | $R \rightsquigarrow \lambda k.\ \|R(k)\|$ |
| **foreign key** | $\forall b, k.\ \|S(b,k)\| \to \|R'(k)\|$ (global assertion) |
| **check** $C$ | $\forall s.\ \|R(s)\| \to C(s)$ |
| $\neq 0$ **encoding** | $\sum_s U \rightsquigarrow \exists s.U$; $U_1 \times U_2 \rightsquigarrow U_1 \wedge U_2$; $U_1 + U_2 \rightsquigarrow U_1 \vee U_2$; $\|U\| \rightsquigarrow U$; $[\,P\,] \rightsquigarrow P$ |
| **ambient assumption** | recursive checks run under $\Phi \wedge (L_1' \vee L_2')$ |

### LIA* (SQLSolver)

$$S^* = \left\{ \vec{v} \;\middle|\; \vec{v} = \sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \right\} \qquad \mathsf{LIA}^*:\ \exists \vec{u}, \vec{v}.\ F_1(\vec{u}, \vec{v}) \wedge \vec{v} \in \{\vec{x} \mid F_2(\vec{x})\}^*$$

Reduction to LIA (Thm. 5.5): replace the closure by $\vec{v} = \sum_{i=1}^{k} \lambda_i \vec{x}_i \wedge \bigwedge_i F_2(\vec{x}_i)$, where the $\vec{x}_i$ are generators of $F_2$. Encoding of U-terms into LIA: $[\,b\,] \mapsto \text{ite}(b, 1, 0)$, $\|E\| \mapsto \text{ite}(E{=}0, 0, 1)$, $\text{not}(E) \mapsto \text{ite}(E{=}0, 1, 0)$, tuple/attribute/table applications $\mapsto$ fresh integer variables.

## B Symbol glossary

| symbol | read as | SQL |
|---|---|---|
| $R(t)$ | multiplicity of row $t$ in $R$ | `FROM R` |
| $[\,P\,]$ | 1 if $P$ else 0 | `WHERE P` |
| $\times$ | multiply multiplicities | `JOIN` |
| $+$ | add multiplicities | `UNION ALL` |
| $\sum_a$ | try every $a$, add up | projection (`COUNT(*)` when applied to a table) |
| $\|\|e\|\|$ | clamp to 0 or 1 | `DISTINCT`, `EXISTS`, `GROUP BY` key |
| $\text{not}(e)$ | 1 iff $e = 0$ | `NOT EXISTS` |
| $\lambda y.\ e$ | anonymous function; here *a bag* | the group feeding an aggregate |
| $\text{Sum}(\lambda y.\ f)$ | $\sum_y y \times f(y)$ | `SUM(y)` |
| $\text{Sum}(\lambda y.\ \|\|f\|\|)$ | each distinct $y$ once | `SUM(DISTINCT y)` |
| $\sum_y \|\|f(y)\|\|$ | count distinct values | `COUNT(DISTINCT y)` |
| QOp, HOp | uninterpreted query / scalar operator | `LIMIT`, `IN`, anything unmodelled |
| $\overline{\mathbb{N}}$ | $\mathbb{N} \cup \{\infty\}$ | the multiplicity domain |
| $\Gamma \mid \Delta \mid \Phi$ | tables \| other symbols \| assumptions | the proving context |

## C Reproducing the measurements

Calcite plans (§2.4, §2.4.2): Calcite 1.19, `HepPlanner` with `AggregateProjectMergeRule` → `AggregateJoinTransposeRule.EXTENDED` → `ProjectMergeRule` → `AggregateProjectMergeRule`. The first `AggregateProjectMergeRule` is required: without it the transpose rule does not fire, because `sql2rel` leaves a `LogicalProject` between the aggregate and the join.

PostgreSQL measurements (Fig. 1): PostgreSQL 18, `EXPLAIN (ANALYZE, VERBOSE, BUFFERS)`; `r` $10^6$ rows with $\sim 10^5$ distinct `x`, `s` $5 \cdot 10^4$ rows with a primary key, both `ANALYZE`d. Equivalence spot-checked on $2 \cdot 10^5$ rows with `EXCEPT ALL` in both directions (0 rows each way).

## References


[1] Shumo Chu, Brendan Murphy, Jared Roesch, Alvin Cheung, Dan Suciu. *Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries.* PVLDB 11(11), 2018. arXiv:1802.02229.

[2] Shuxian Wang, Sicheng Pan, Alvin Cheung. *QED: A Powerful Query Equivalence Decider for SQL.* PVLDB 17(11):3602–3614, 2024.

[3] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, Jinyang Li. *Proving Query Equivalence Using Linear Integer Arithmetic.* Proc. ACM Manag. Data 1(4), Article 227 (SIGMOD), 2023.

[4] Qi Zhou, Joy Arulraj, Shamkant Navathe, William Harris, Jinpeng Wu. *SPES: A Symbolic Approach to Proving Query Equivalence Under Bag Semantics.* ICDE 2022.

[5] Todd J. Green, Grigoris Karvounarakis, Val Tannen. *Provenance Semirings.* PODS 2007.

[6] Sara Cohen. *Equivalence of Queries Combining Set and Bag-Set Semantics.* PODS 2006.

[7] Ruzica Piskac, Viktor Kuncak. *Decision Procedures for Multisets with Cardinality Constraints.* VMCAI 2008.

[8] Maxwell Levatich, Nikolaj Bjørner, Ruzica Piskac, Sharon Shoham. *Solving LIA* Using Approximations.* VMCAI 2020.

[9] Zhaoguo Wang et al. *WeTune: Automatic Discovery and Verification of Query Rewrite Rules.* SIGMOD 2022.